[BUSINESS] FORECASTING AND PREDICTIVE MODELING

LECTURE 4

Lect. PhD. Oneț-Marian Zsuzsanna

Babeș - Bolyai University
Computer Science and Mathematics Faculty

2025 - 2026

In Lecture 3 ...

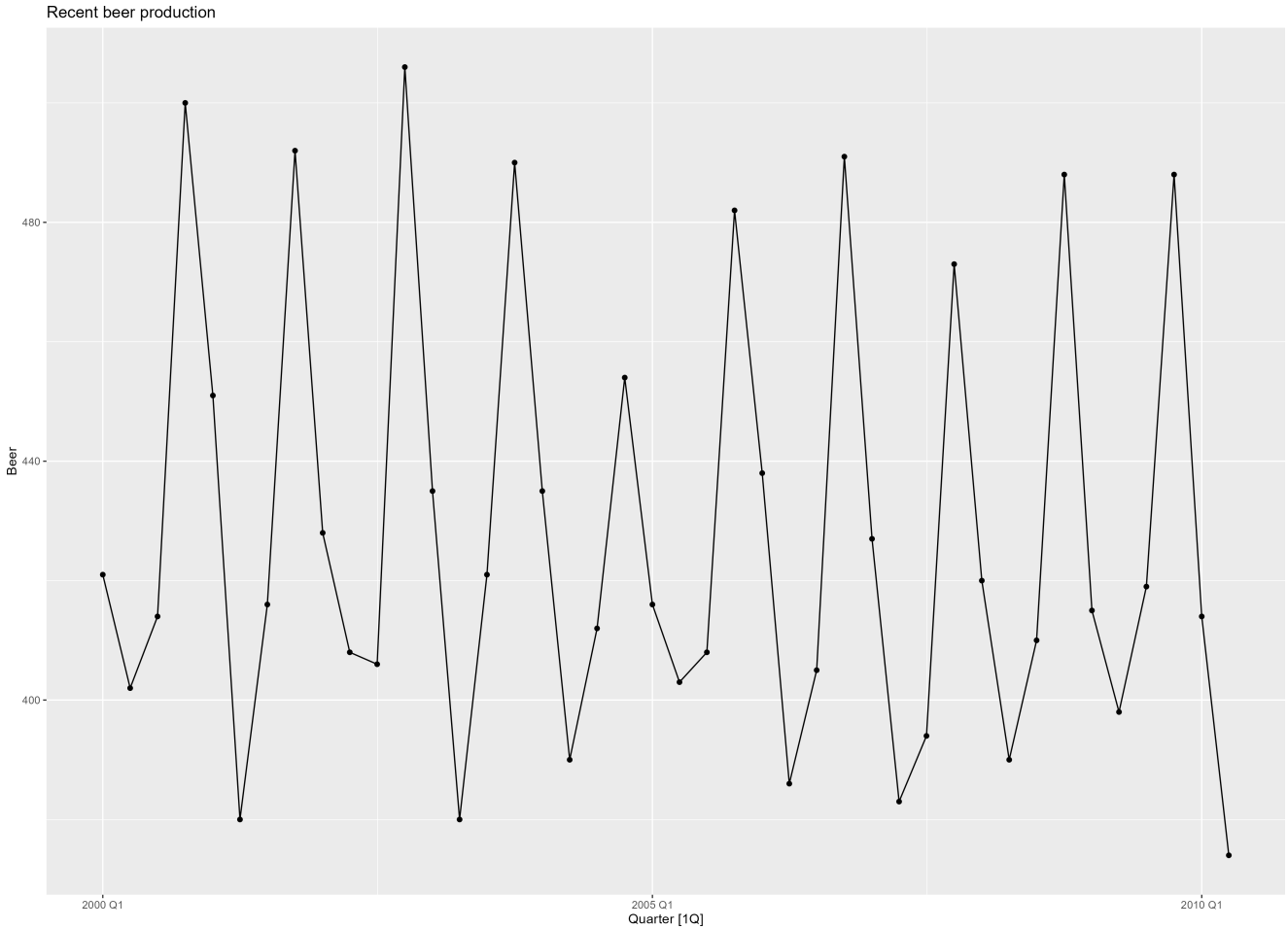
- Time series visualization
  - Line plot
  - Seasonal plot
  - Seasonal subseries plot
  - Scatter plot
  - Lag plot

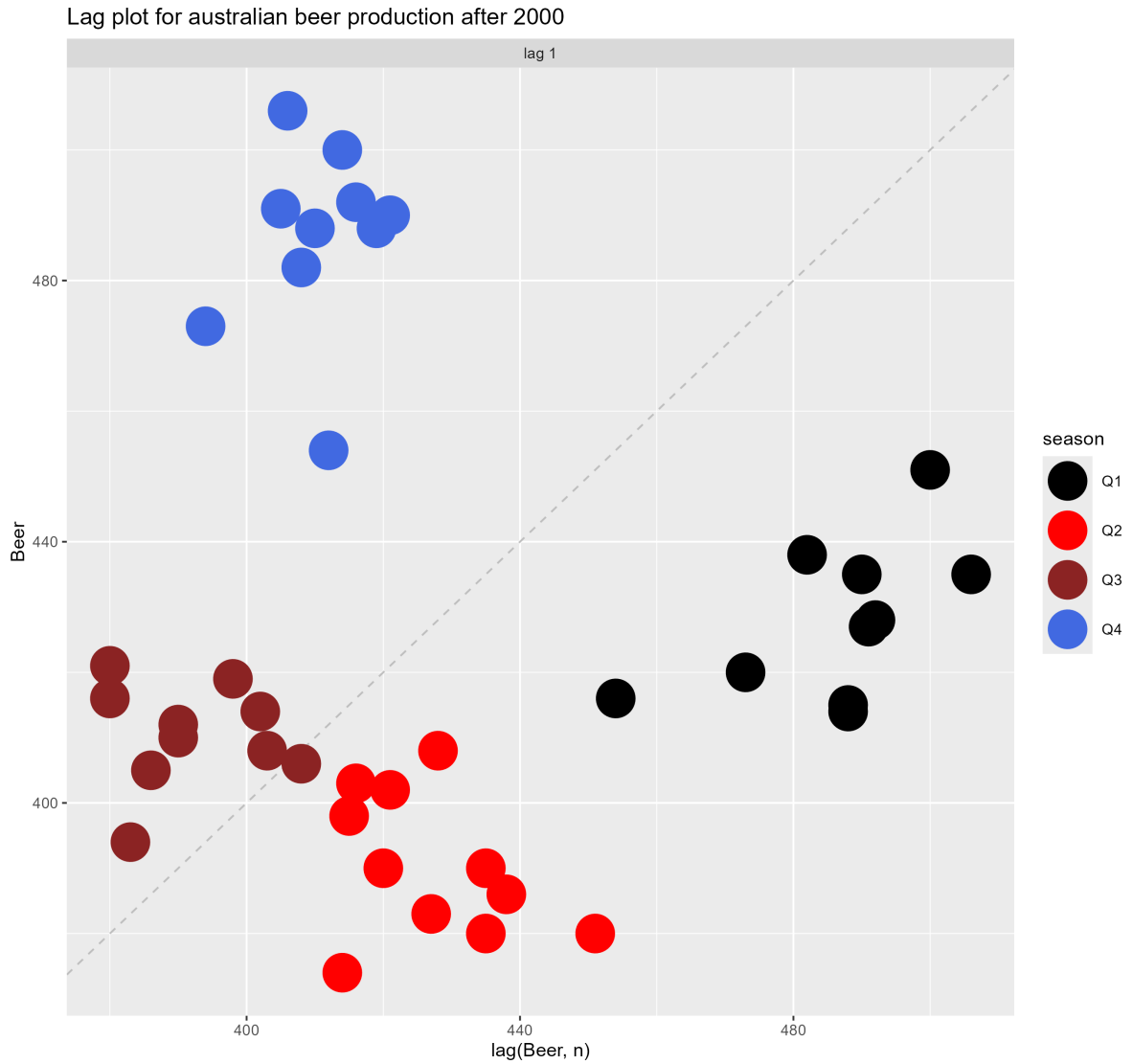
Today

- Time series visualization
  - Autocorrelation plot
- Stationarity
- Differencing

Lag plots - Recap

- *Lag plots* are scatter plots, where the time series y_t is plotted against y_{t-k} for different values of k.
- For example, for $k = 1$ we plot the value of y_t against y_{t-1}.
- In R a lag plot can be created using the function *gg_lag*


Recent beer production
Beer
480
440
400
2000 Q1
2005 Q1
2010 Q1
Quarter [1Q]


Lag plot for australian beer production after 2000
lag 1
Beer
480
440
400
400
440
480
lag(Beer, n)
season
Q1
Q2
Q3
Q4

Autocorrelation

- Correlation values can be computed for lagged values, they are called *autocorrelation*. You can have an autocorrelation coefficient for each possible value of k: for example, r_1 is the correlation between y_t and y_{t-1} (lag of 1).

Computing the autocorrelation

- The formula for the Pearson correlation coefficient between variables X and Y is the following:

$$r_{XY} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i = \bar{y})^2}}$$

- The actual way of computing the autocorrelation is the following:

$$r_k = \frac{\sum_{t=k+1}^{T}(y_t - \bar{y})(y_{t-k} - \bar{y})}{\sum_{t=1}^{T}(y_t - \bar{y})^2}$$

- Are the two formulae equivalent?

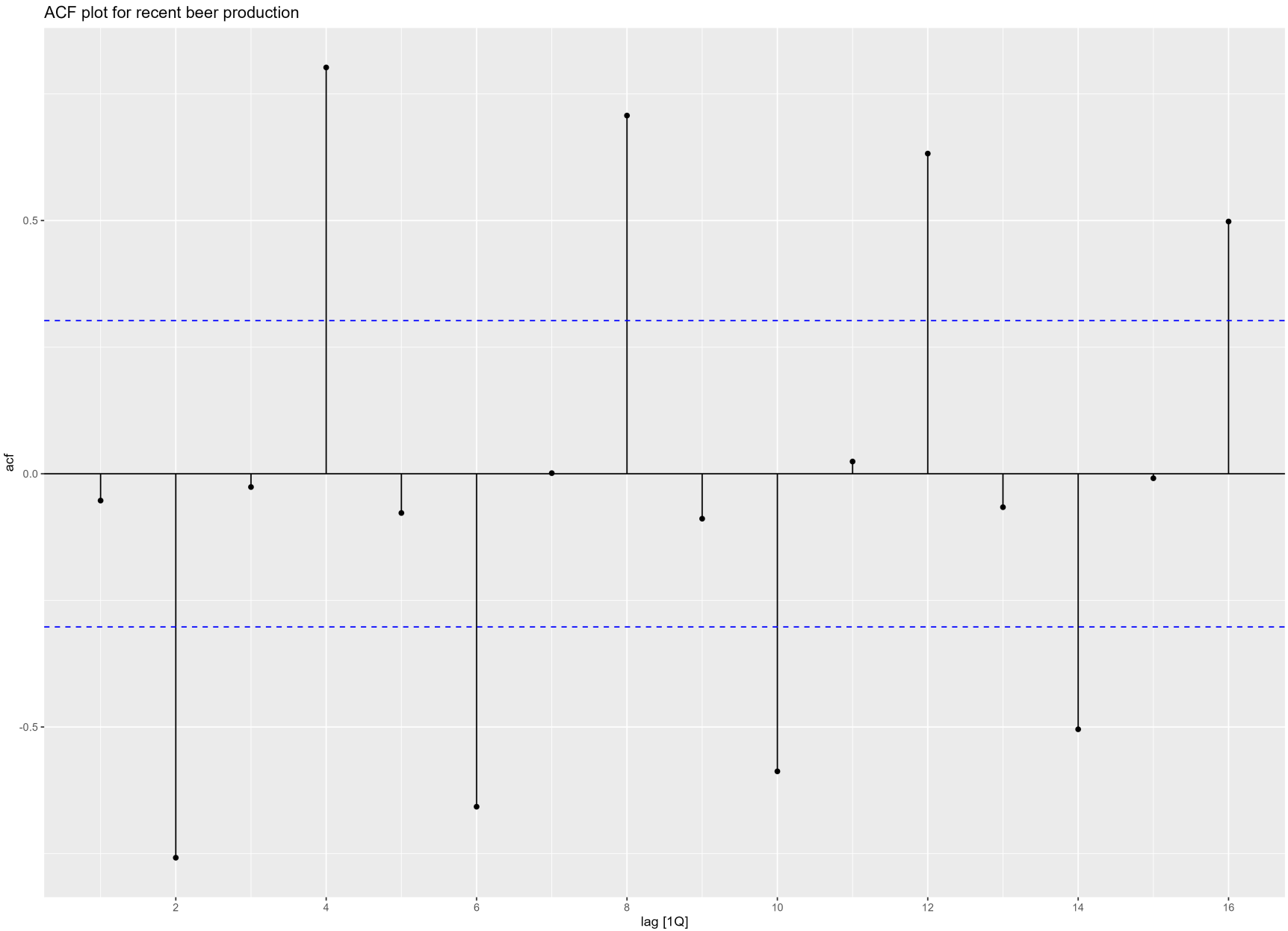
Autocorrelation

- In R you can compute the autocorrelation values using the *ACF* function. By default, the function computes the values up to a lag of $10 * log_{10}(N/m)$ (where N is the number of observations and m is the number of series - in our case 1), but this value can be set using the *lag_max* parameter.
- The result of *ACF* is a time series, which can be *autoplot* for a visual representation.

```
recent_beer <- aus_production |>
  filter(year(Quarter) >= 2000) |>
  select(Quarter, Beer)

recent_beer |> ACF()
class(recent_beer |> ACF())
recent_beer |> ACF() |> as_tsibble(index = lag) |> autoplot(acf)
recent_beer |> ACF() |> autoplot()
recent_beer |> ACF() |> autoplot() + geom_point()
```

- **Obs.** When we look at the class of the result from the ACF function, its first type is *tbl_cf*, which is needed to make sure that *autoplot* plots it in a different ways, not like a regular time series.


ACF plot for recent beer production
acf
0.5
0.0
-0.5
2
4
6
8
10
12
14
16
lag [1Q]

- We have high positive correlations for lags 4, 8, etc. because of the seasonality of the data.
- We have high negative correlation for lags 2, 6, 10, etc.
- The blue dashed lines will be explained a little later.

Another example of autocorrelation

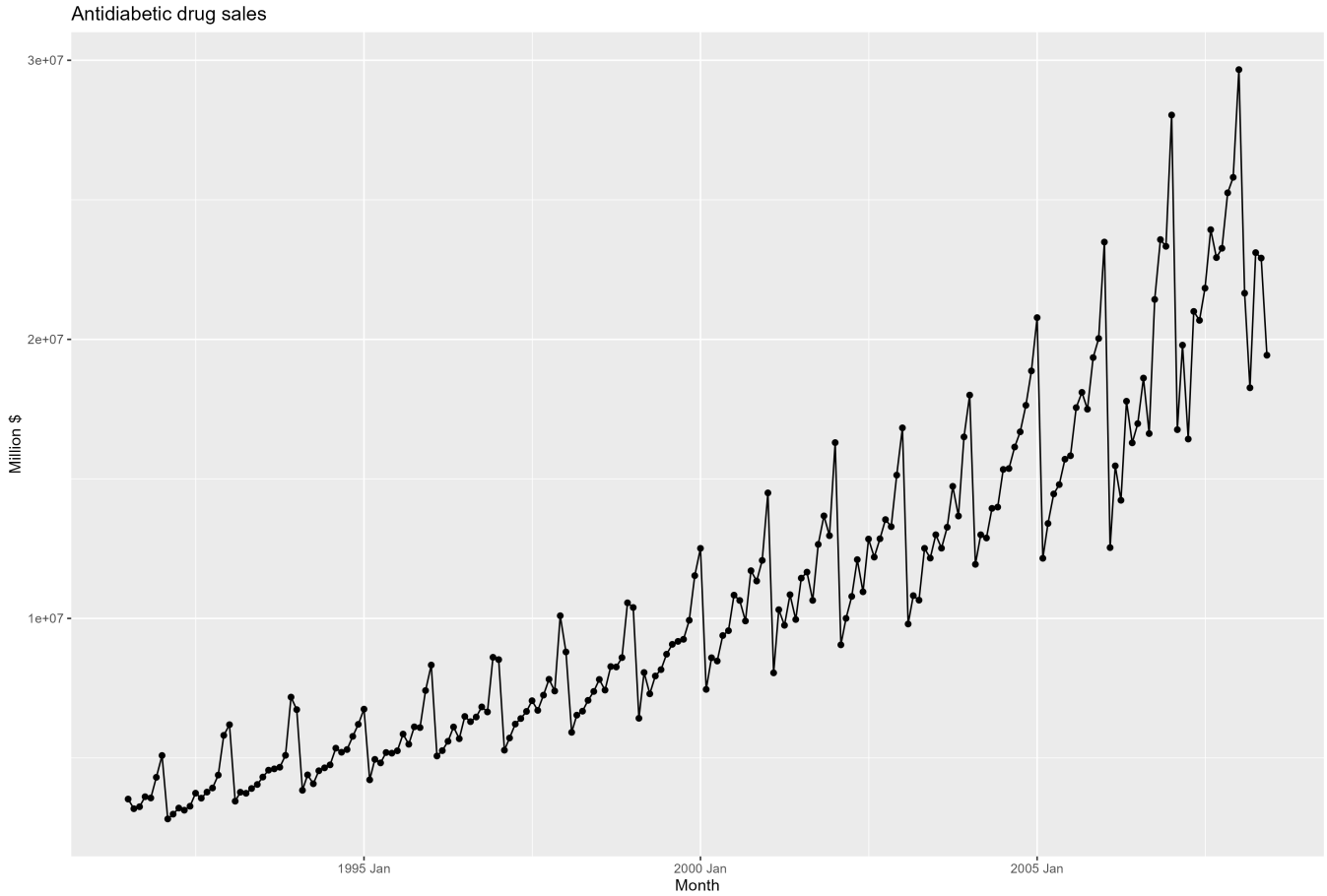
- Let us see the autocorrelation for the antidiabetic drug time series (A10).

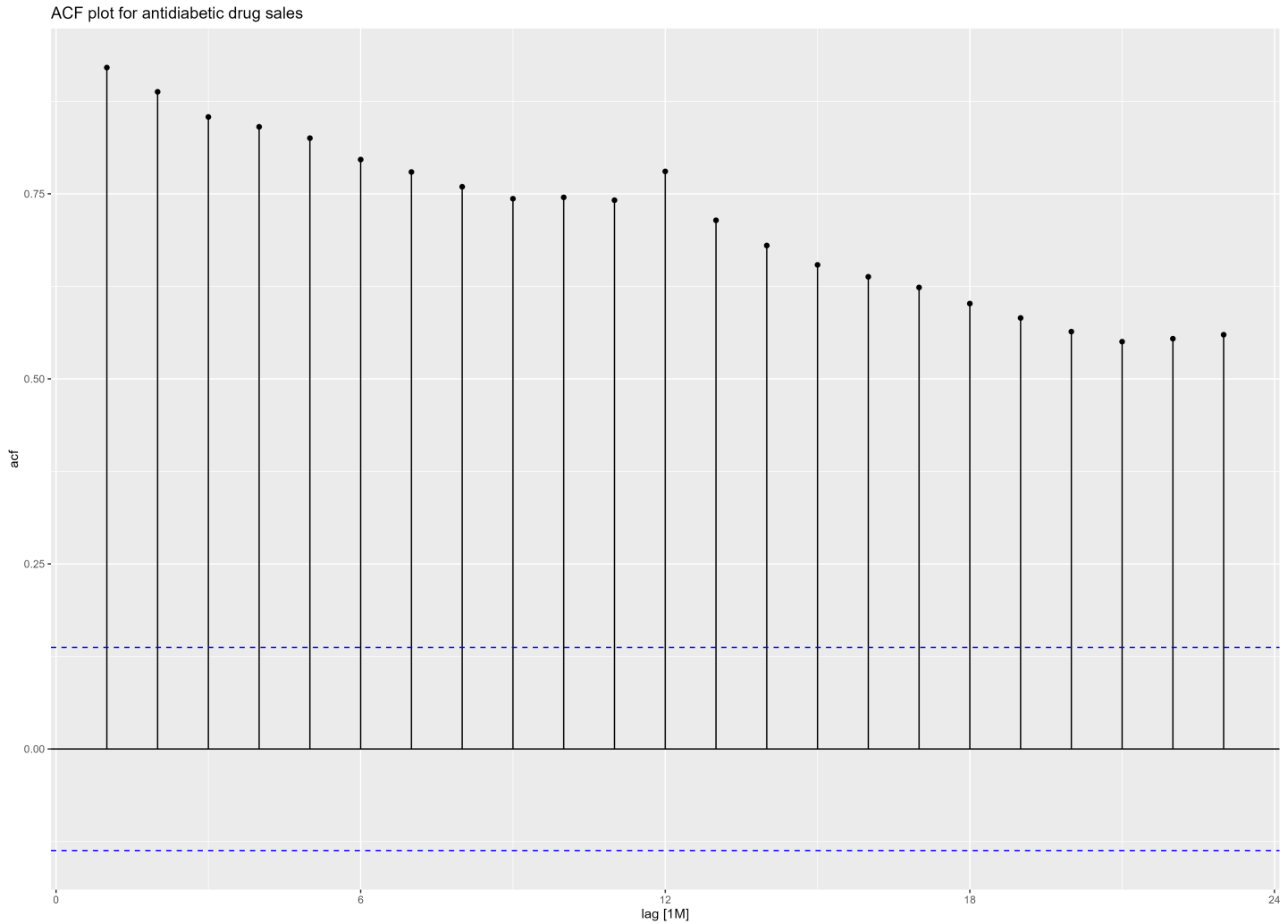
- The beer time series contained quarterly data, a10 is monthly, so we might need more lags to see some pattern. This can be set with the *lag_max* parameter.

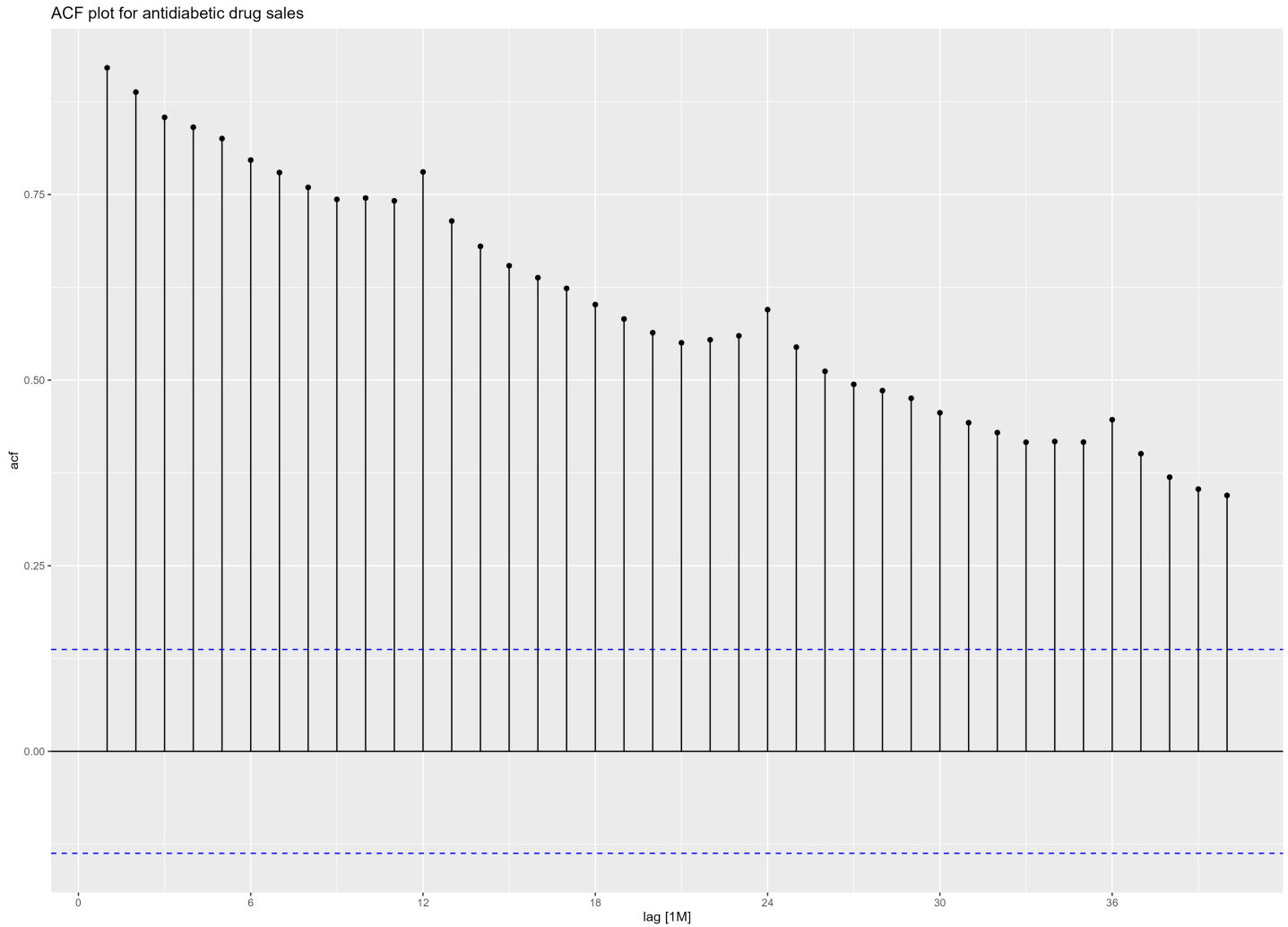
```
a10 |> autoplot() +
  geom_point() +
  labs(title = "Antidiabetic drug sales")

a10 |> ACF() |>
  autoplot() +
  geom_point() +
  labs(title = "ACF plot for antidiabetic drug sales")

a10 |> ACF(lag_max = 40) |>
  autoplot() + geom_point() +
  labs(title = "ACF plot for antidiabetic drug sales")
```


Antidiabetic drug sales
3e+07
2e+07
1e+07
Million $
1995 Jan
2000 Jan
2005 Jan
Month


ACF plot for antidiabetic drug sales
acf
0.75
0.50
0.25
0.00
0
6
12
18
24
lag [1M]



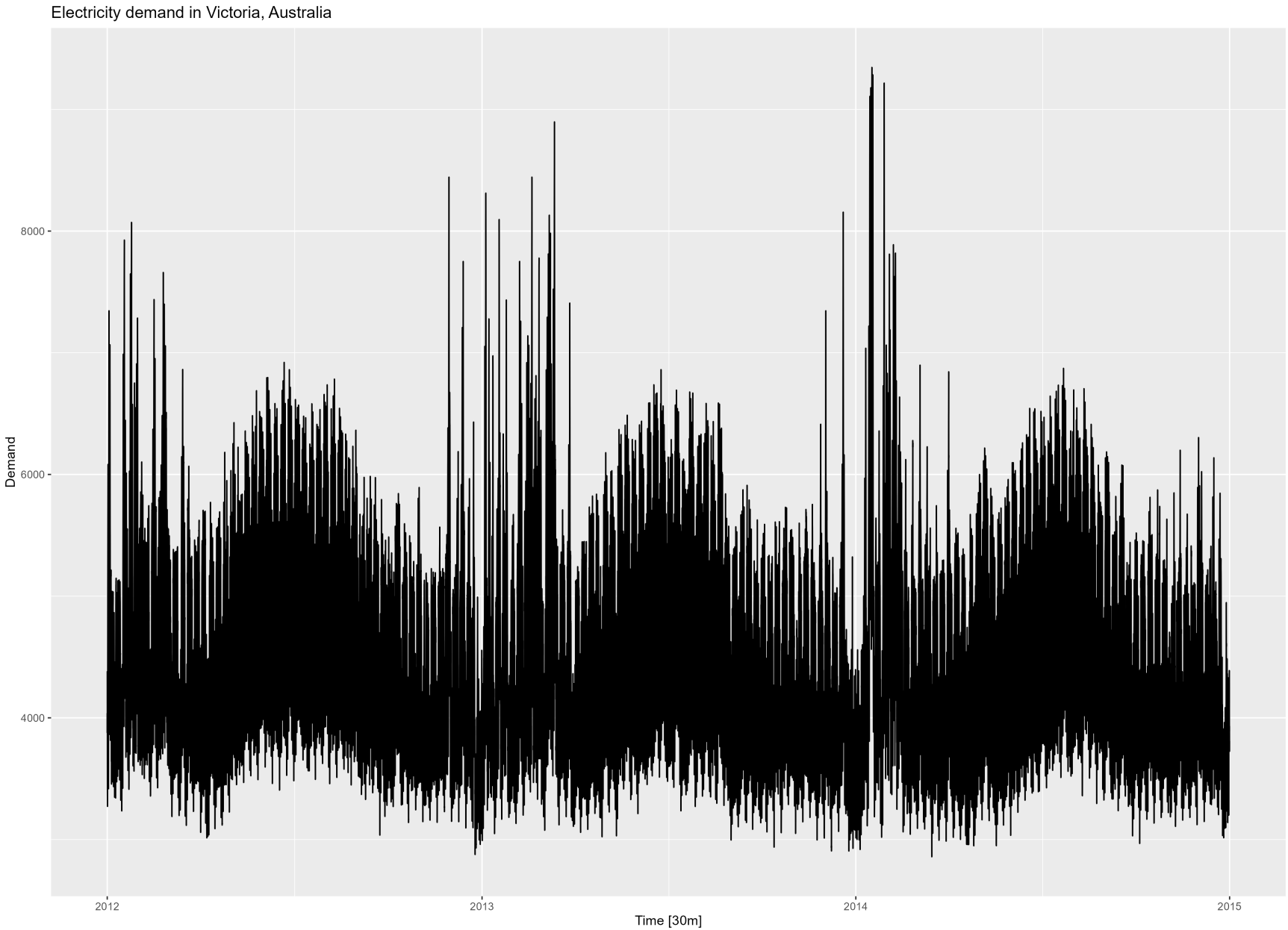
- What do you notice on this plot?

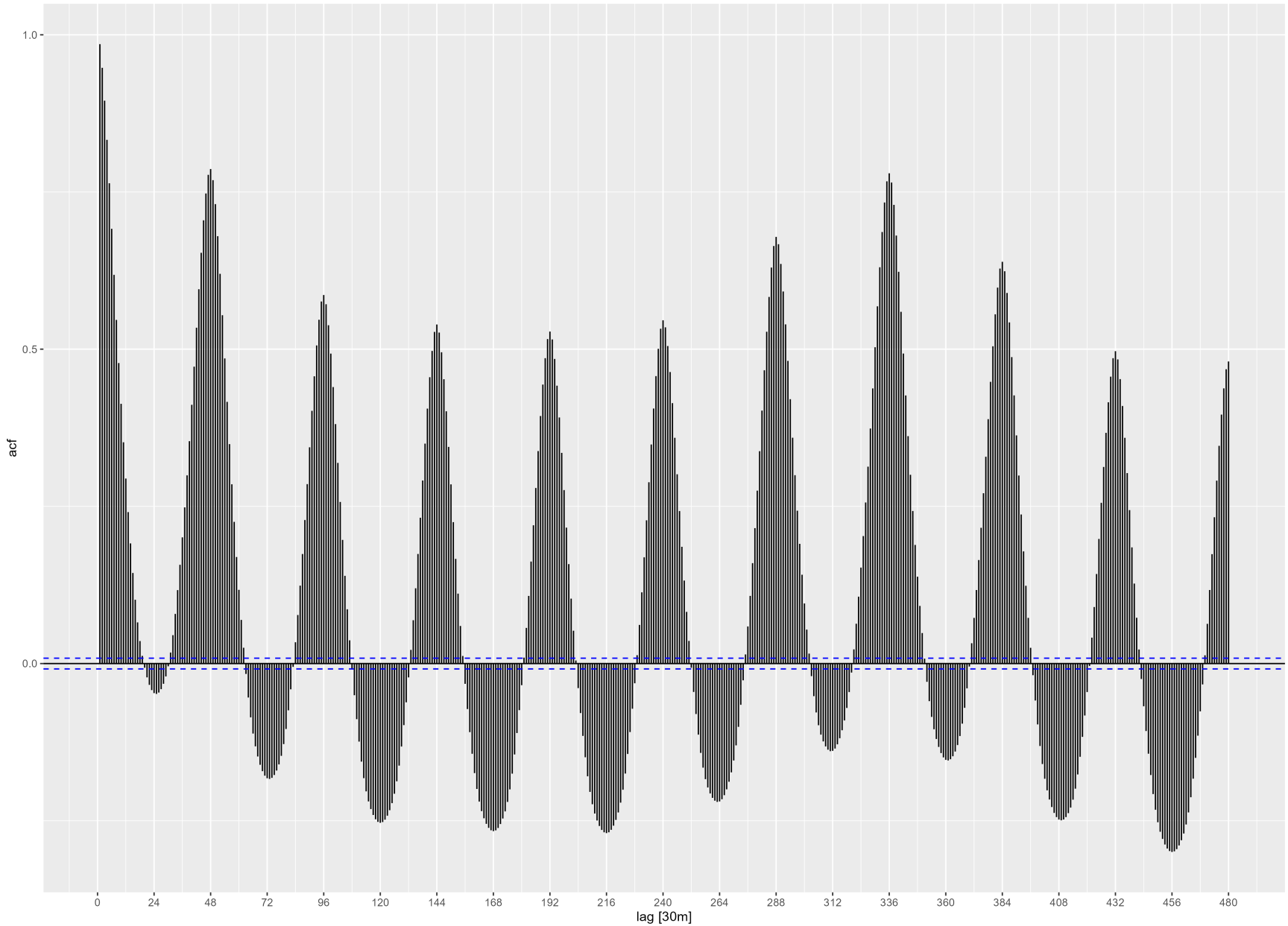
Autocorrelation for trends and seasonality

- In case of seasonal data, the autocorrelation will be higher for lags which are multiple of the frequency (see previous example with beer production).
- In case of trended data, the autocorrelation should be high for smaller lags (because data points close to each other have similar values). This autocorrelation decreases as the value of the lag increases.

```
vic_elec |>autoplot() +
  labs(title= "Electricity demand in Victoria, Australia")

vic_elec |> ACF(Demand, lag_max = 480) |> autoplot() +
labs("ACF plot for electricity demand in Victoria")
```


Electricity demand in Victoria, Australia
Demand
8000
6000
4000
2012
2013
2014
2015
Time [30m]


1.0
0.5
0.0
acf
0
24
48
72
96
120
144
168
192
216
240
264
288
312
336
360
384
408
432
456
480
lag [30m]

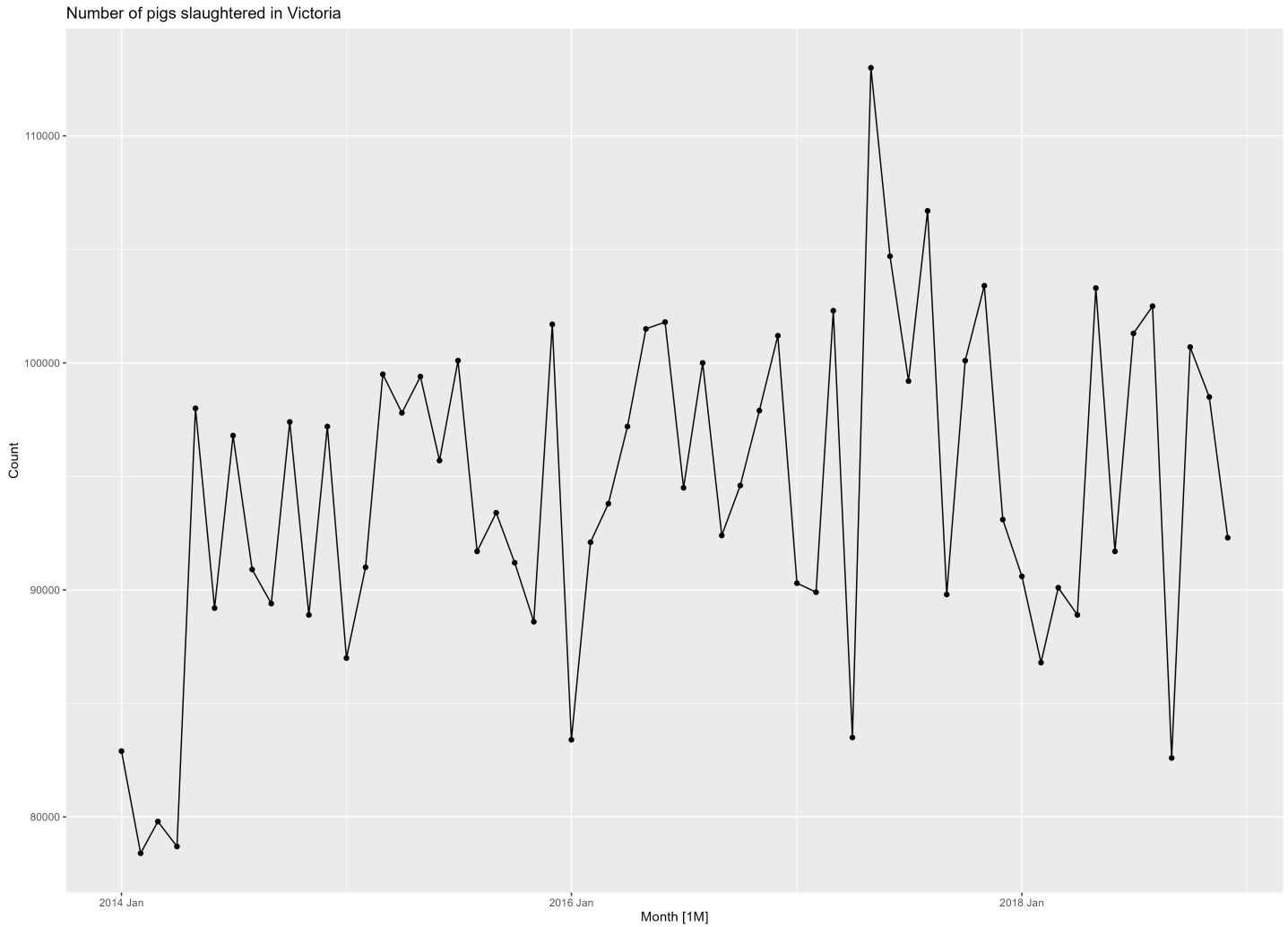
- Let us consider another example, a subset of the time series *aus_livestock*.

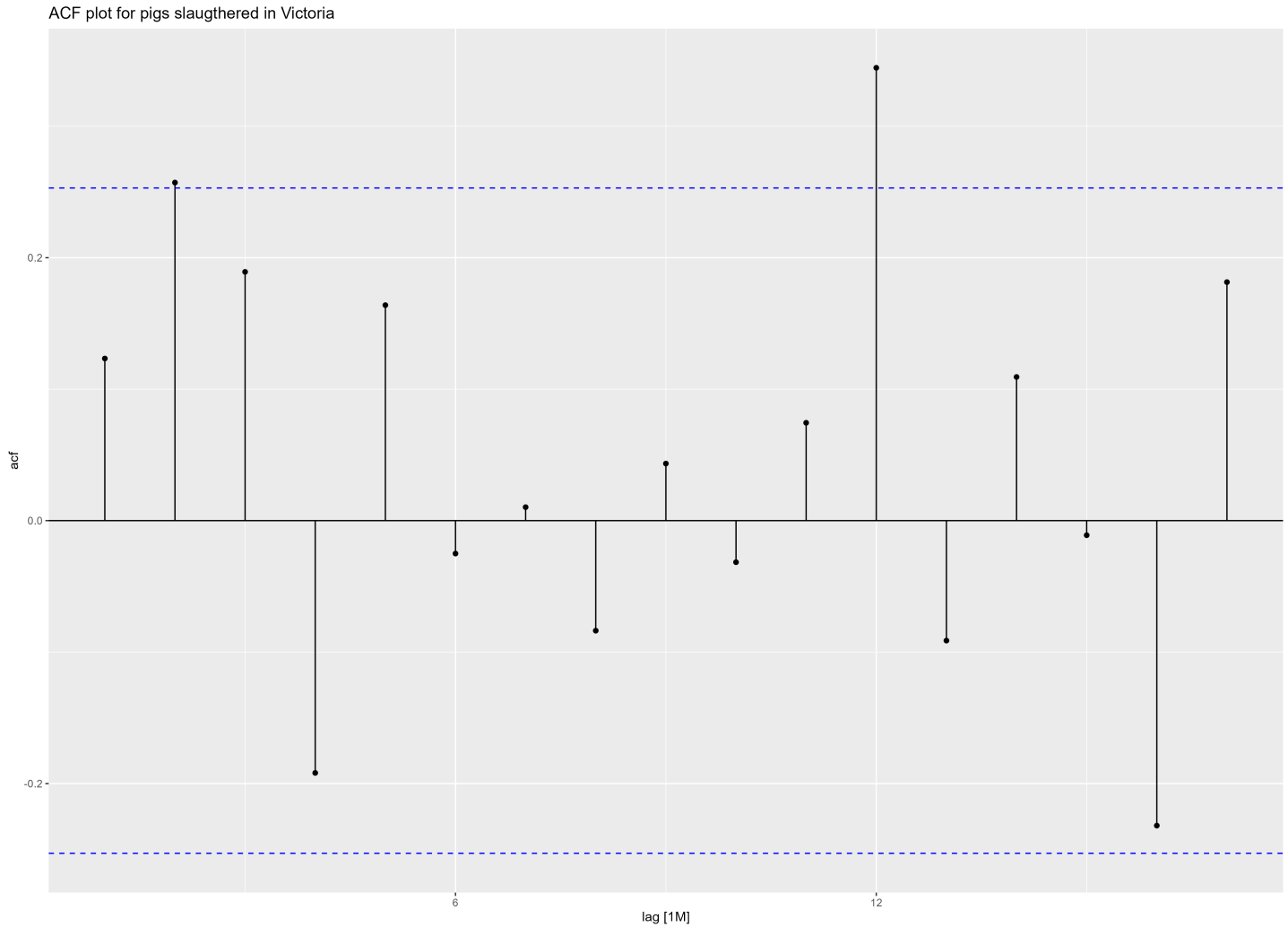
```
aus_livestock |>
  filter(Animal == "Pigs", State == "Victoria", year(Month) >= 2014) -> aus_pigs

aus_pigs |> autoplot() +
  geom_point() +
  labs(title = "Number of pigs slaughtered in Victoria")

ggsave("TimeSeriesPlot_AusPigs.png")

aus_pigs |>
  ACF() |>
  autoplot() +
  geom_point() +
  labs(title = "ACF plot for pigs slaugthered in Victoria")
```


Number of pigs slaughtered in Victoria
Count
110000
100000
90000
80000
2014 Jan
2016 Jan
2018 Jan
Month [1M]



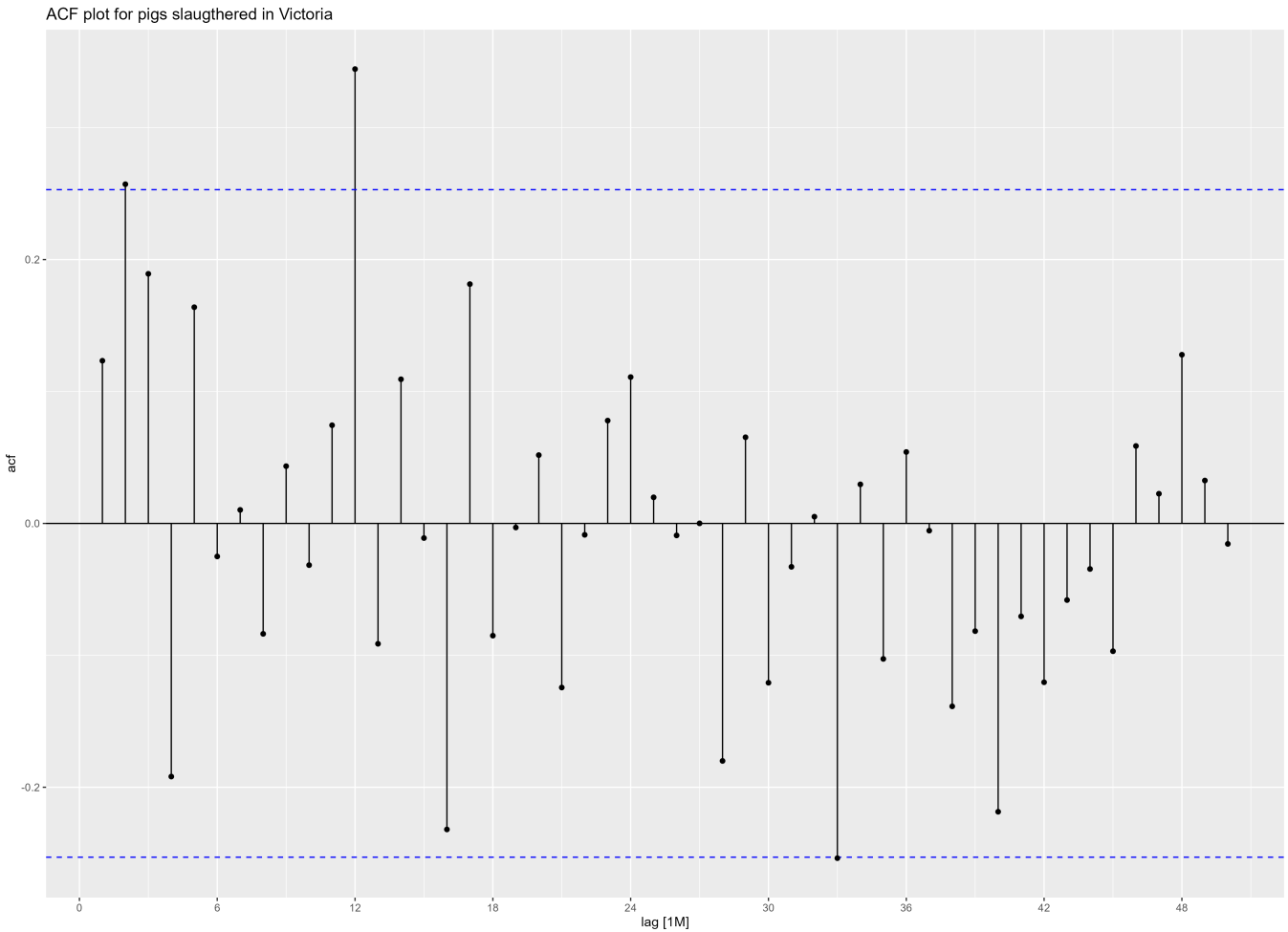
- What do you notice in the ACF plot?

White noise

- A time series where the autocorrelation is close to 0 for most lag values is called *white noise*.
- More precisely, we expect 95% of autocorrelation values to be within the values $\pm\frac{1.96}{\sqrt{T}}$, where T is the number of observations.
- The values $\pm\frac{1.96}{\sqrt{T}}$ are plotted as the blue lines on the ACF plot.
- When all autocorrelation values are between the blue lines or at most 5% of the values are outside of them, the data is probably white noise.

- The *pigs* time series has 2 autocorrelations outside the blue lines: one at lag 12 and a very small one at lag 2.

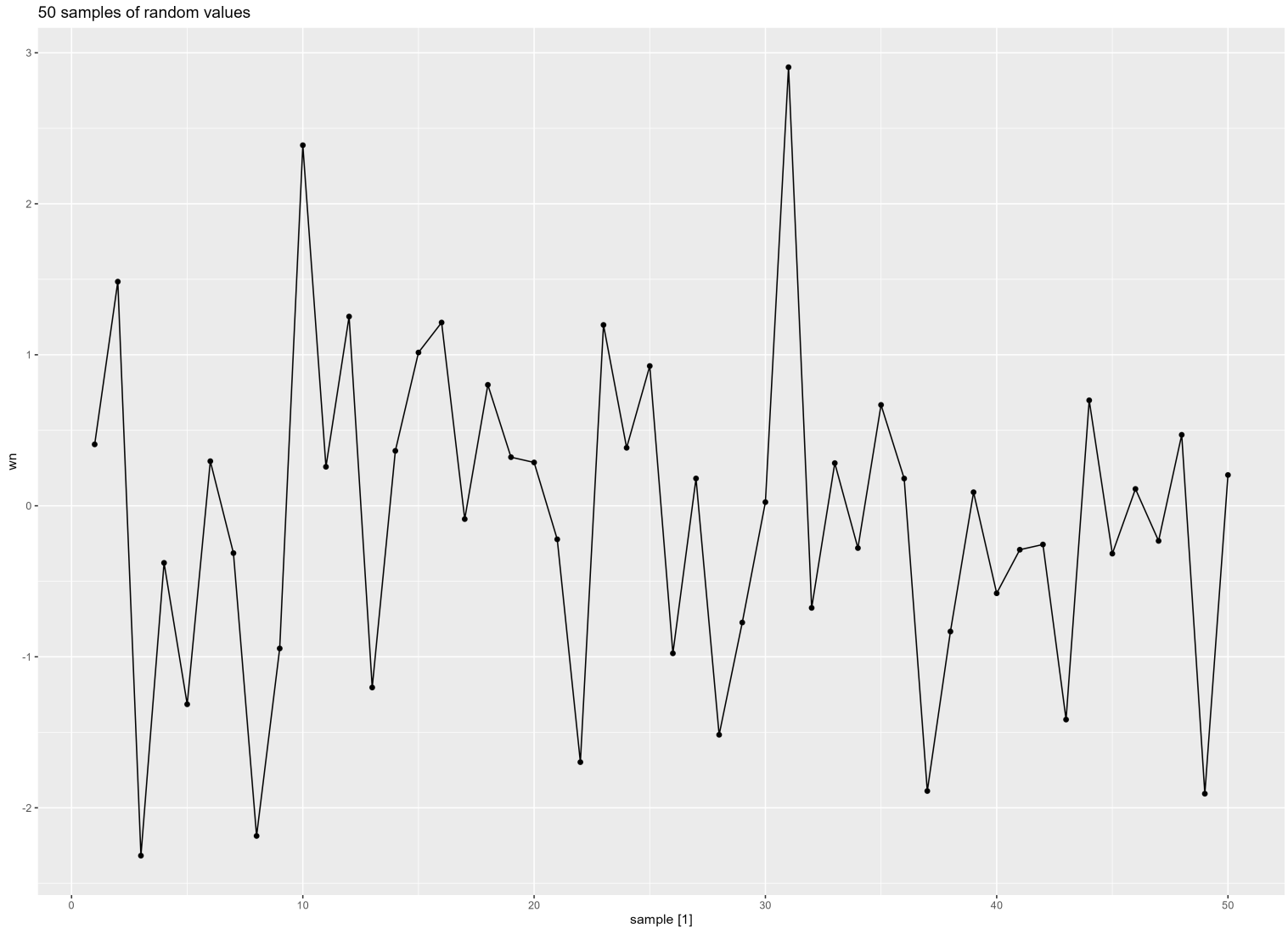
- Since we only have 16 lags, this suggests that the data is not white noise. Moreover, since the largest lag is at 12, it might actually show some kind of seasonality. Let us look at more autocorrelations for the pigs data.

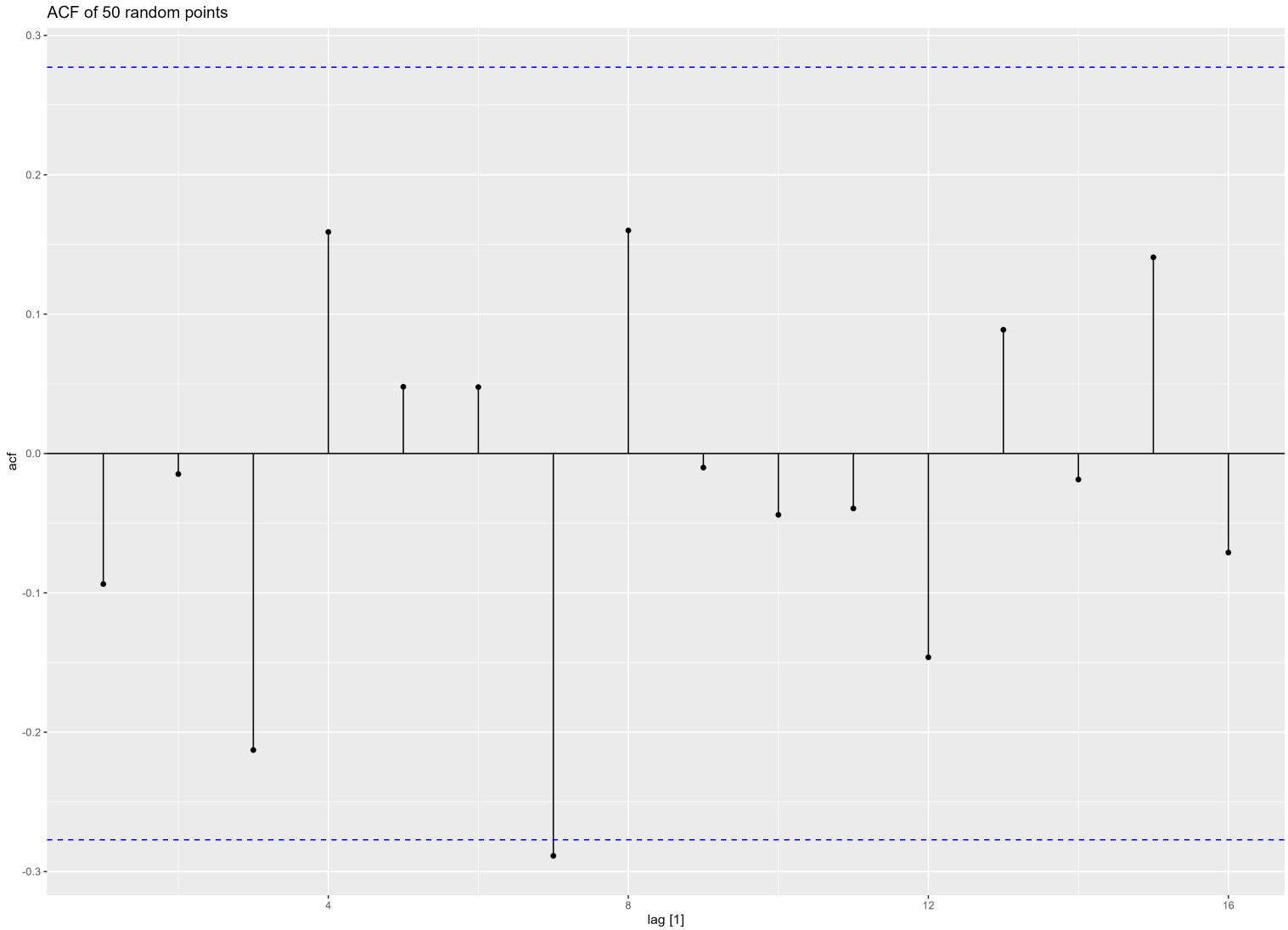

ACF plot for pigs slaugthered in Victoria
acf
0.2
0.0
-0.2
0
6
12
18
24
30
36
42
48
lag [1M]

- Let us generate a time series with randomly chosen normal values.

```
set.seed(39)
wn <- tsibble(sample = 1:50, wn = rnorm(50), index =  sample)
wn |> autoplot() +
  geom_point() +
  labs(title = "50 samples of random values")


wn |> ACF() |>
  autoplot() +
  geom_point() +
  labs(title = "ACF of 50 random points")
```


50 samples of random values
wn
3
2
1
0
-1
-2
0
10
20
30
40
50
sample [1]


ACF of 50 random points
acf
0.3
0.2
0.1
0.0
-0.1
-0.2
-0.3
4
8
12
16
lag [1]

Stationary time series

- A time series is *stationary* if its observations are not dependent on time and summary statistics computed for it (ex. mean and variance) are not dependent on time.
- In other words, a time series is stationary, if
  - it has no trend (it is roughly horizontal)
  - it has constant variance (it is homoscedastic)
  - it has no predictable patterns (no seasons).
- **Obs.** white noise is stationary (but not every stationary time series is white noise).
- Some forecasting methods require time series to be stationary.

- Actually, the reason why autocorrelation is computed with the sample mean instead of the two different means, is that it is assumed that the time series is stationary, so its mean does not depend on where do you sample from (i.e., the mean of all samples should be approximately the same as the mean of the lagged observations)

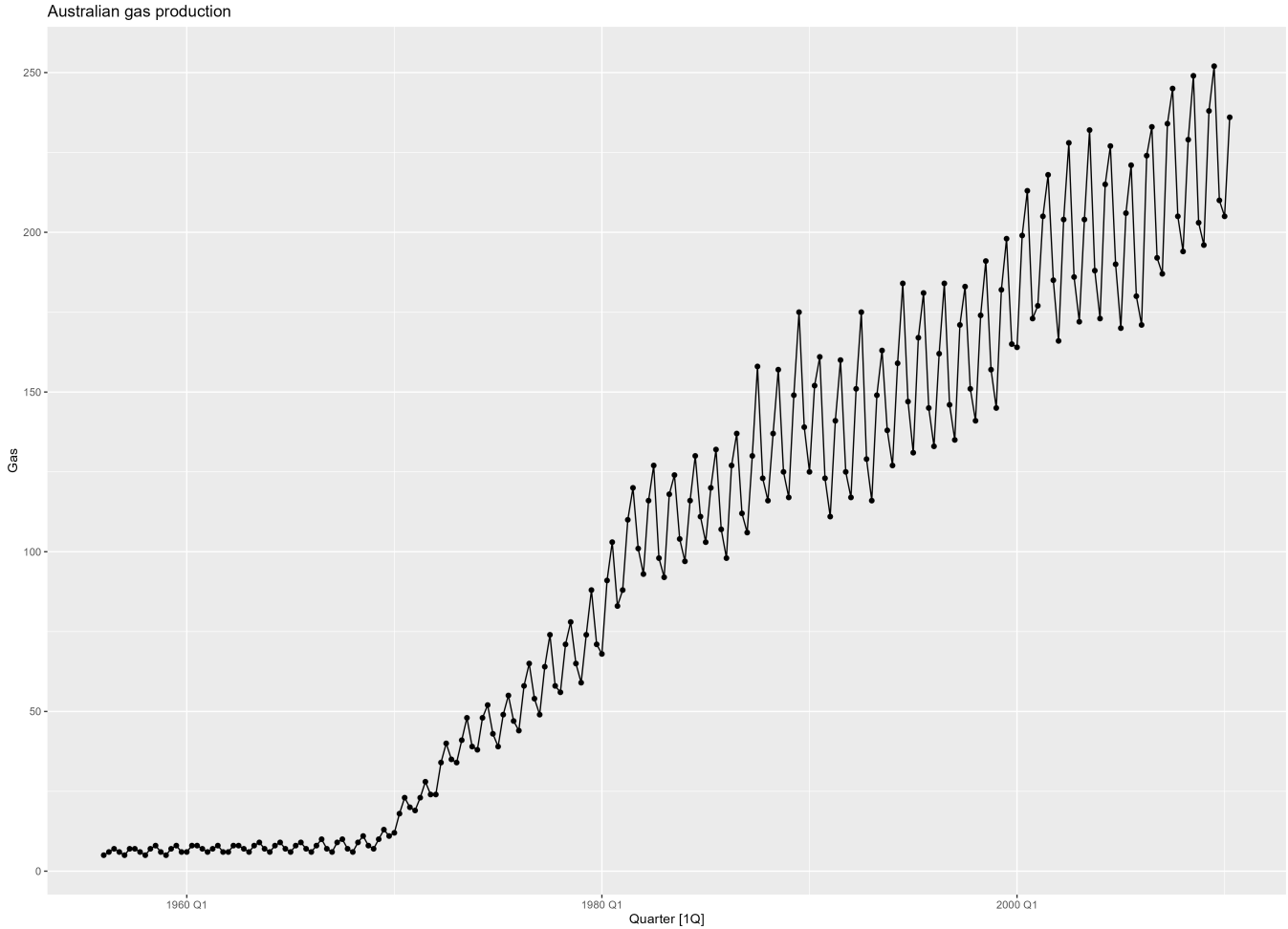
- This does not mean that we should not compute autocorrelation for non-stationary data, we just need to be careful when interpreting the ACF plot, because high autocorrelation can be the result of a trend (not some other pattern in data).

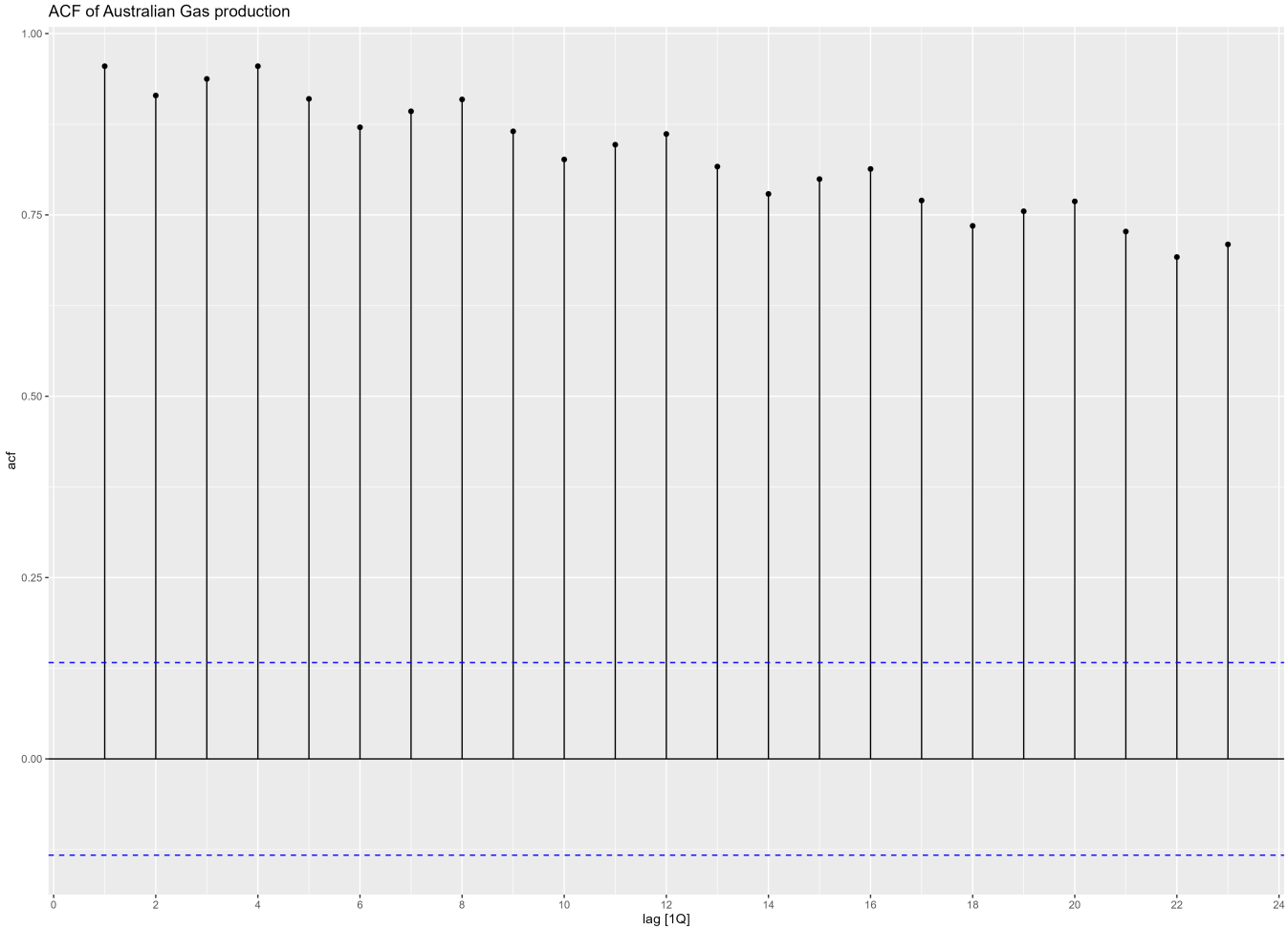
- Let's consider an example which is clearly not stationary

```
aus_gas <- aus_production |>
  select(Quarter, Gas)

aus_gas |> autoplot() +
  geom_point() +
  labs(title= "Australian gas production")

aus_gas |> ACF() |>
  autoplot() +
  geom_point() +
  labs(title = "ACF of Australian Gas production")
```


Australian gas production
Gas
250
200
150
100
50
0
1960 Q1
1980 Q1
2000 Q1
Quarter [1Q]

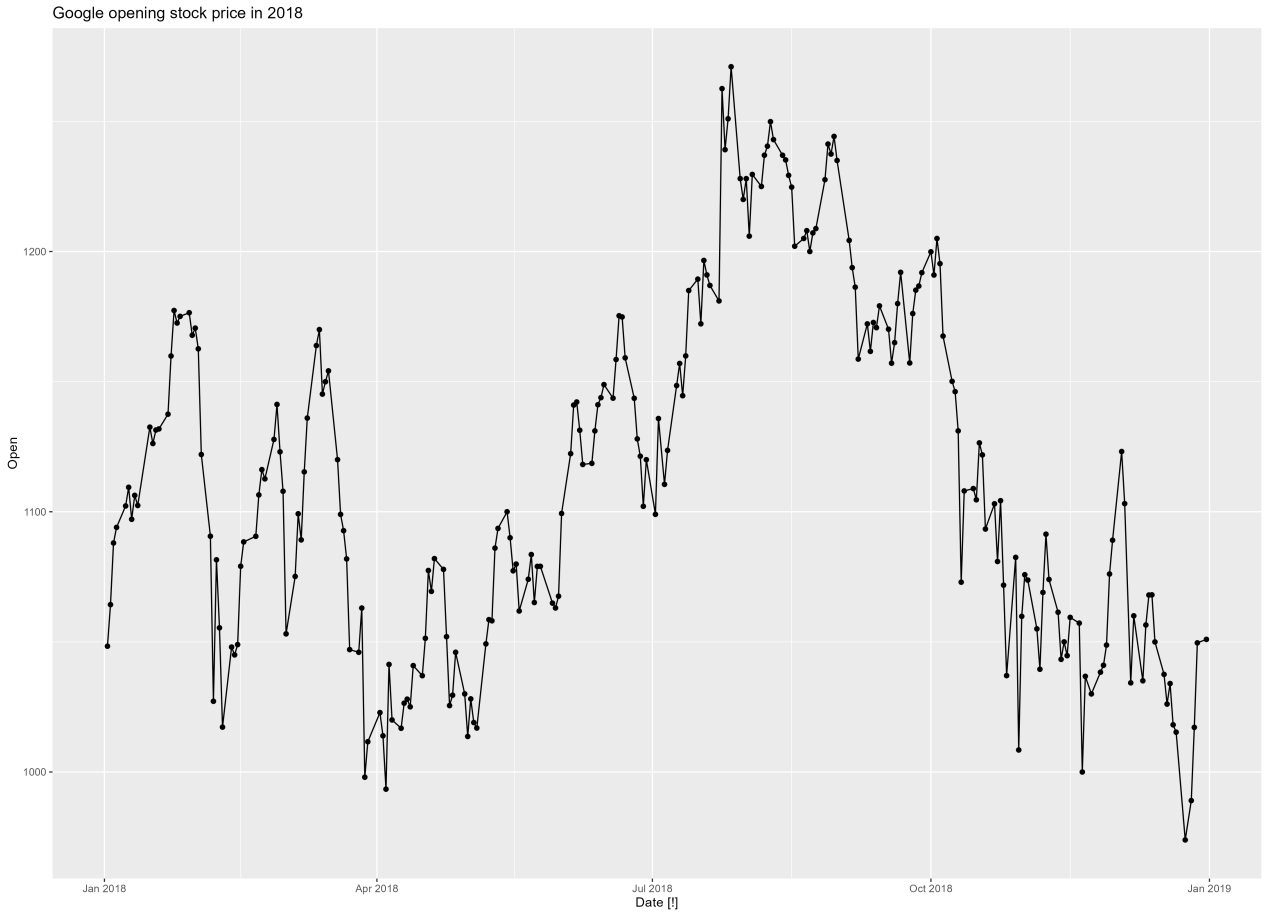

ACF of Australian Gas production
1.00
0.75
0.50
0.25
0.00
acf
0
2
4
6
8
10
12
14
16
18
20
22
24
lag [1Q]

- Stationary time series have ACF plots which drop quickly to zero, while in case of non-stationary time series the decrease in general is gradual. For non-stationary data the first autocorrelation also tends to be a large, positive value.

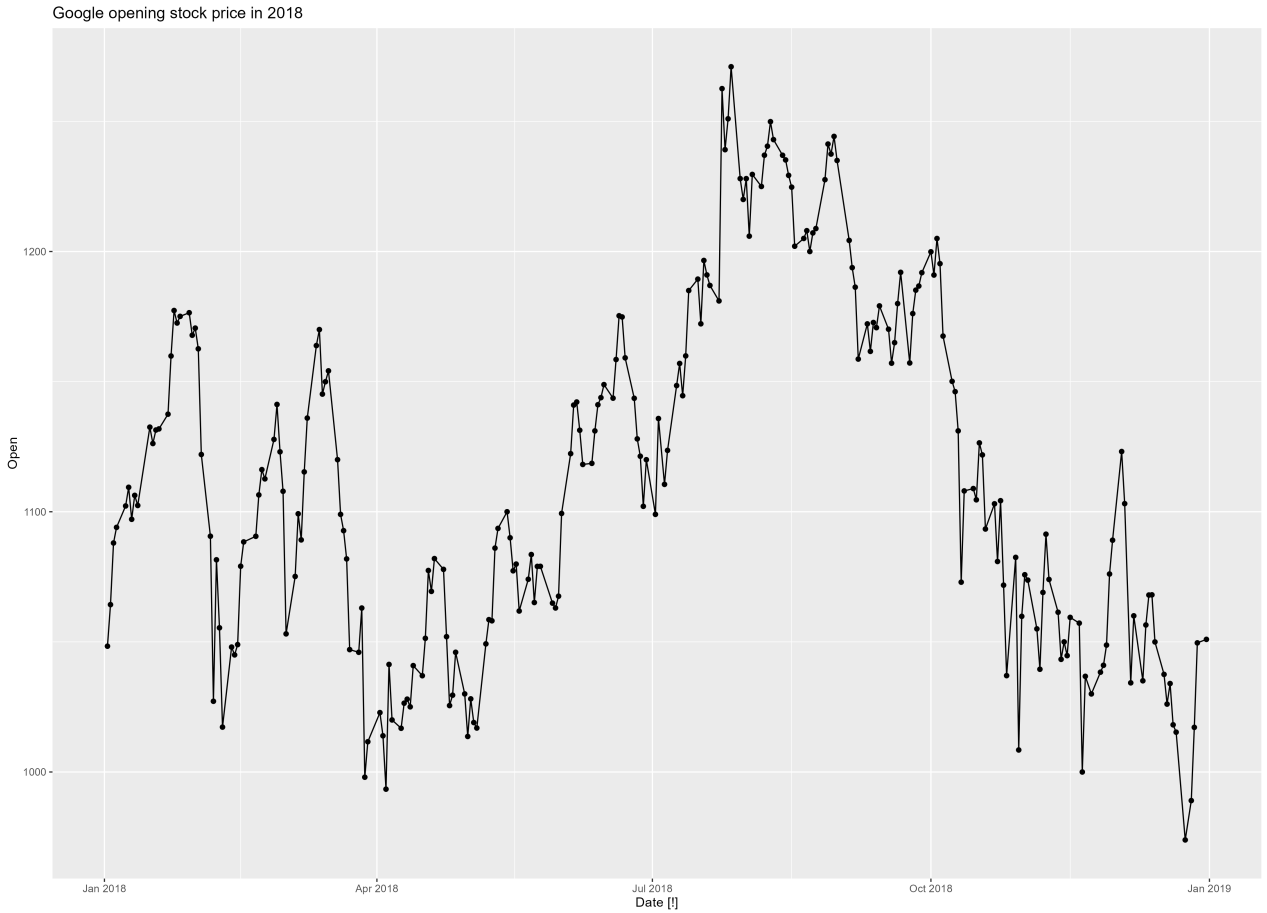
```
google_2018 <- gafa_stock |>
  filter(Symbol == "GOOG", year(Date) == 2018)

google_2018 |>
  autoplot() +
  geom_point() +
  labs(title = "Google opening stock price in 2018")

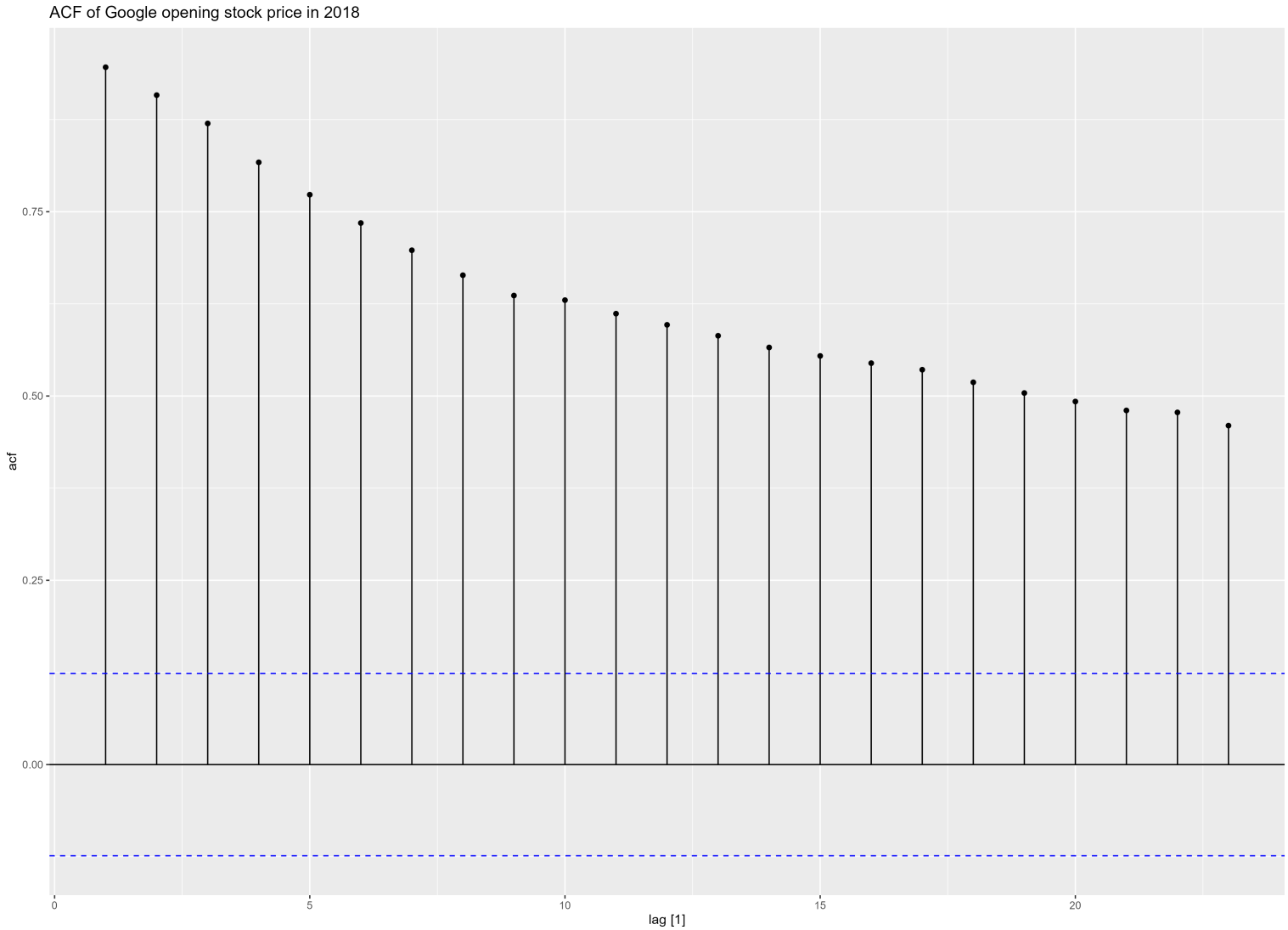
google_2018 |>
  ACF() |>
  autoplot() +
  geom_point() +
  labs(title = "ACF of Google opening stock price in 2018")
```



- Is this time series stationary?



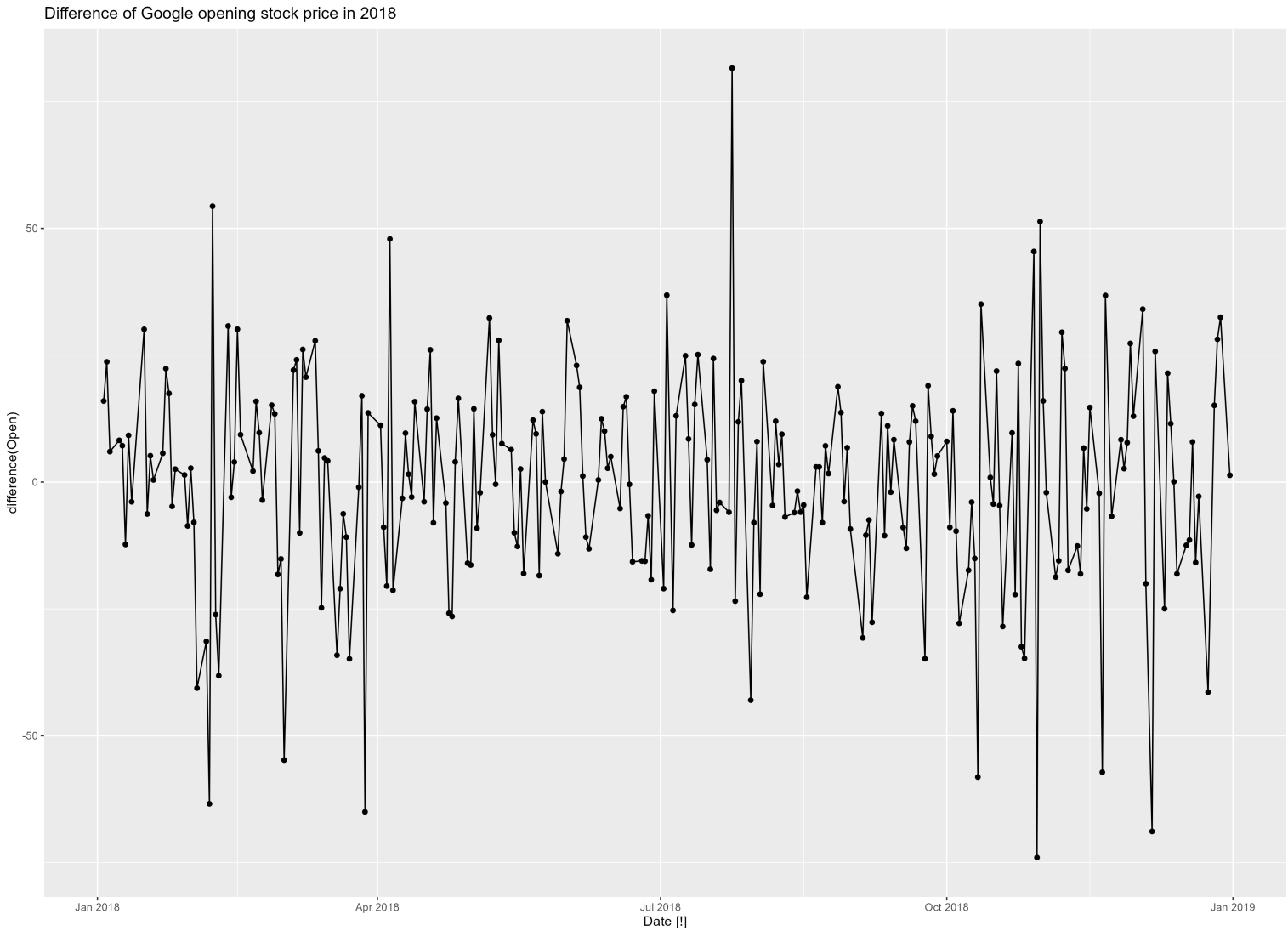
- Is this time series stationary?
- While it might not be so obvious, this is not stationary either (even if we do not have seasons or trend).

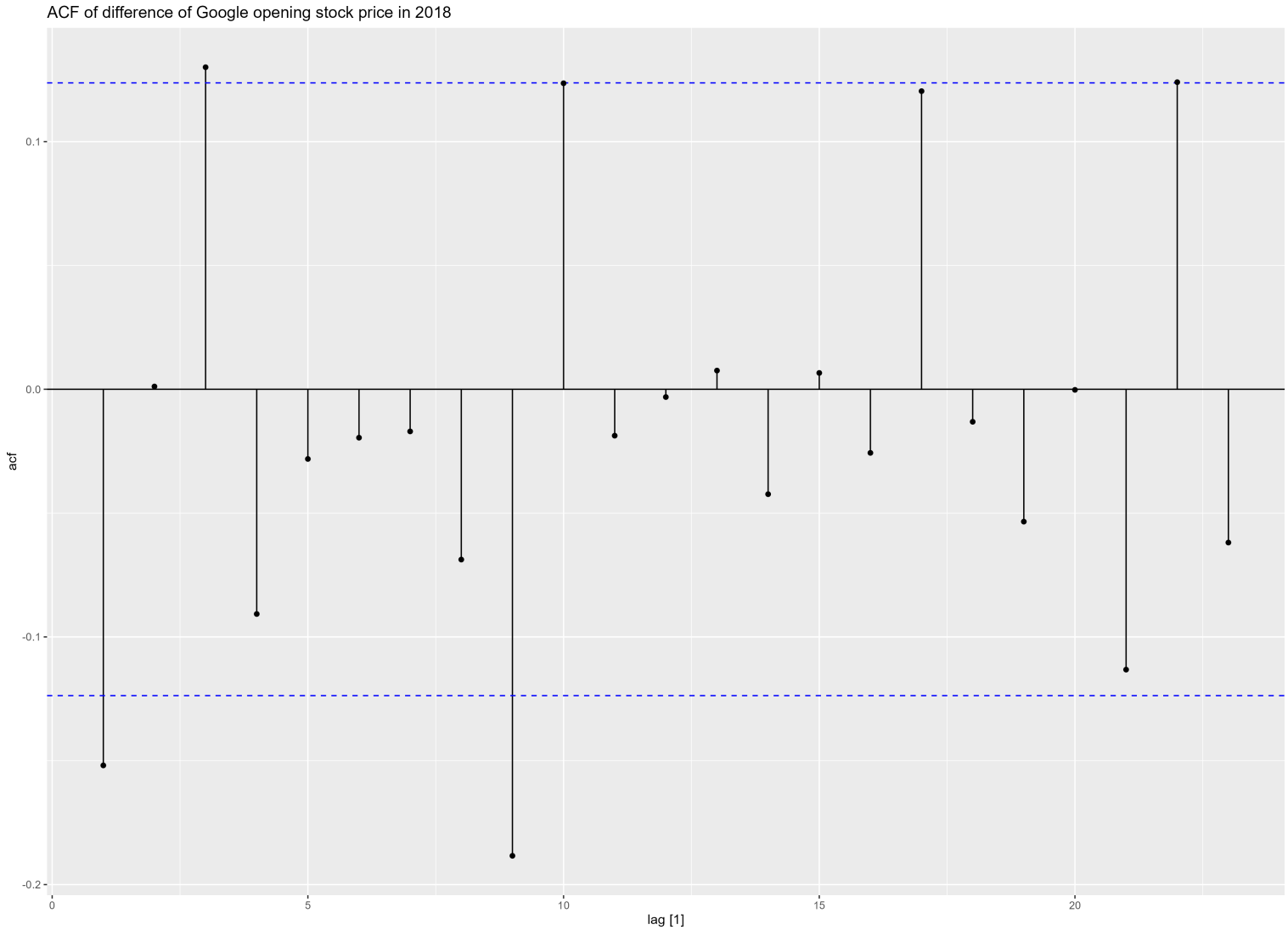

ACF of Google opening stock price in 2018
acf
0.75
0.50
0.25
0.00
0
5
10
15
20
lag [1]

- Let's look at the plot and ACF of the difference of consecutive observations.

```
google_2018 |>
  autoplot(difference(Open)) +
  geom_point() +
  labs(title = "Difference of Google opening stock price in 2018")

google_2018 |>
  ACF(difference(Open)) |>
  autoplot() +
  geom_point() +
  labs(title = "ACF of difference of Google opening stock price in 2018")
```


Difference of Google opening stock price in 2018
50
0
-50
difference(Open)
Jan 2018
Apr 2018
Jul 2018
Oct 2018
Jan 2019
Date [!]


ACF of difference of Google opening stock price in 2018
0.1
0.0
-0.1
-0.2
acf
0
5
10
15
20
lag [1]

Statistical tests

- While in some cases it is very simple to identify whether you have trend and/or season from the plot of the data (and conclude that it is not stationary), sometimes it might be more complicated (or you just need a method that can be automated for a large number of time series).
- We can use statistical tests:
  - *Augmented Dickey-Fuller* test - ADF test (H_0 : non-stationary)
  - *Kwiatkowski-Philips-Schmidt-Shin* - KPSS test (H_0 : stationary) - this is preferred in forecasting, since we only want to transform if there is strong evidence that we need to do it.

The Kwiatkowski-Philips-Schmidt-Shin (KPSS) test

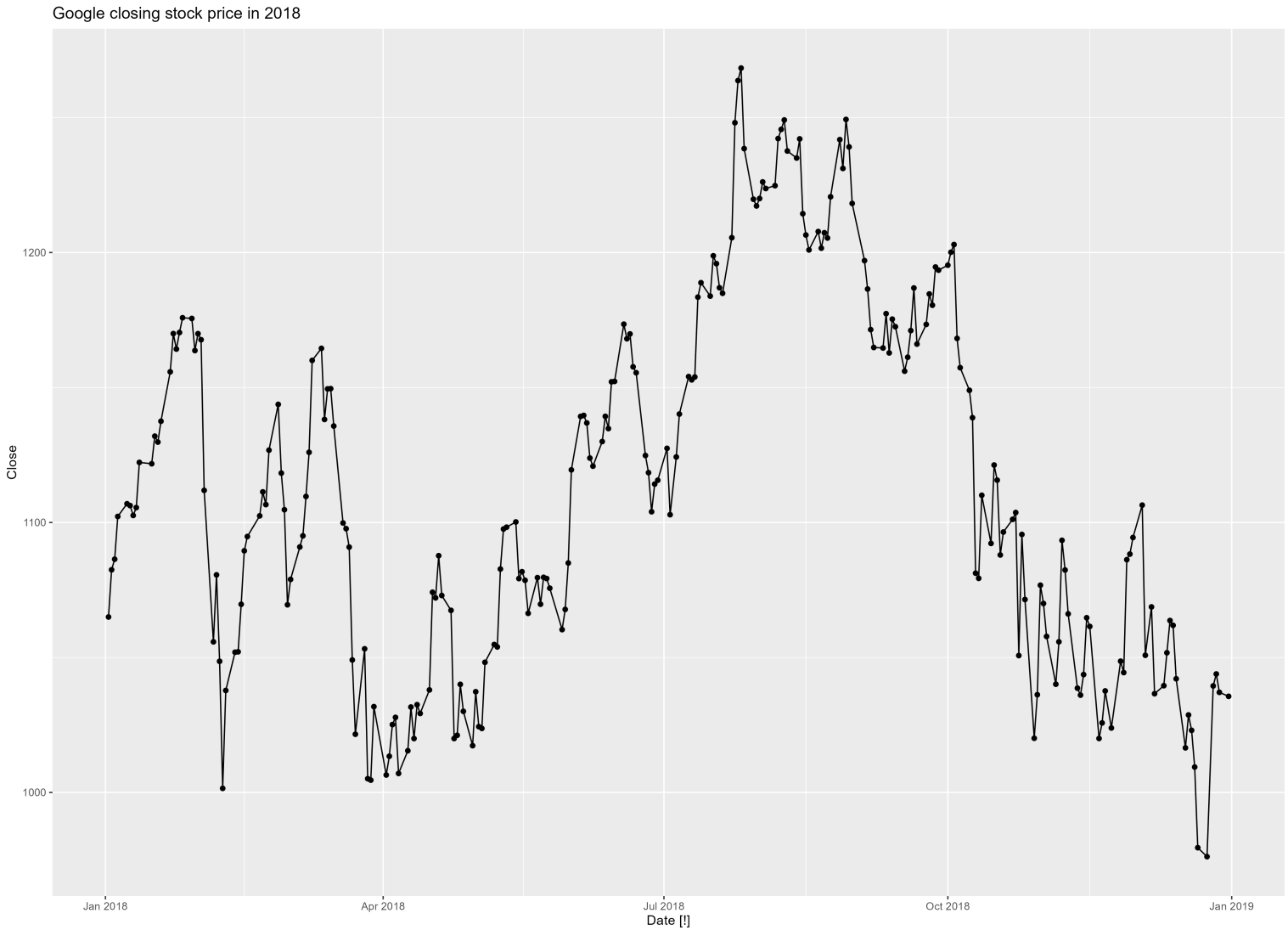
- The KPSS test is a *stationarity test*.
- **The null hypothesis:** The time series is stationary
- **The alternative hypothesis:** The time series is not stationary (it contains something called a *unit root*)
- Interpretation: If the p-value is less than the pre-defined significance level (in general 0.05) the null hypothesis is rejected and the time series is considered to be non-stationary.
- The KPSS test can be run in R using the *features* function and passing it as parameter that we want to compute the *unitroot_kpss* feature).

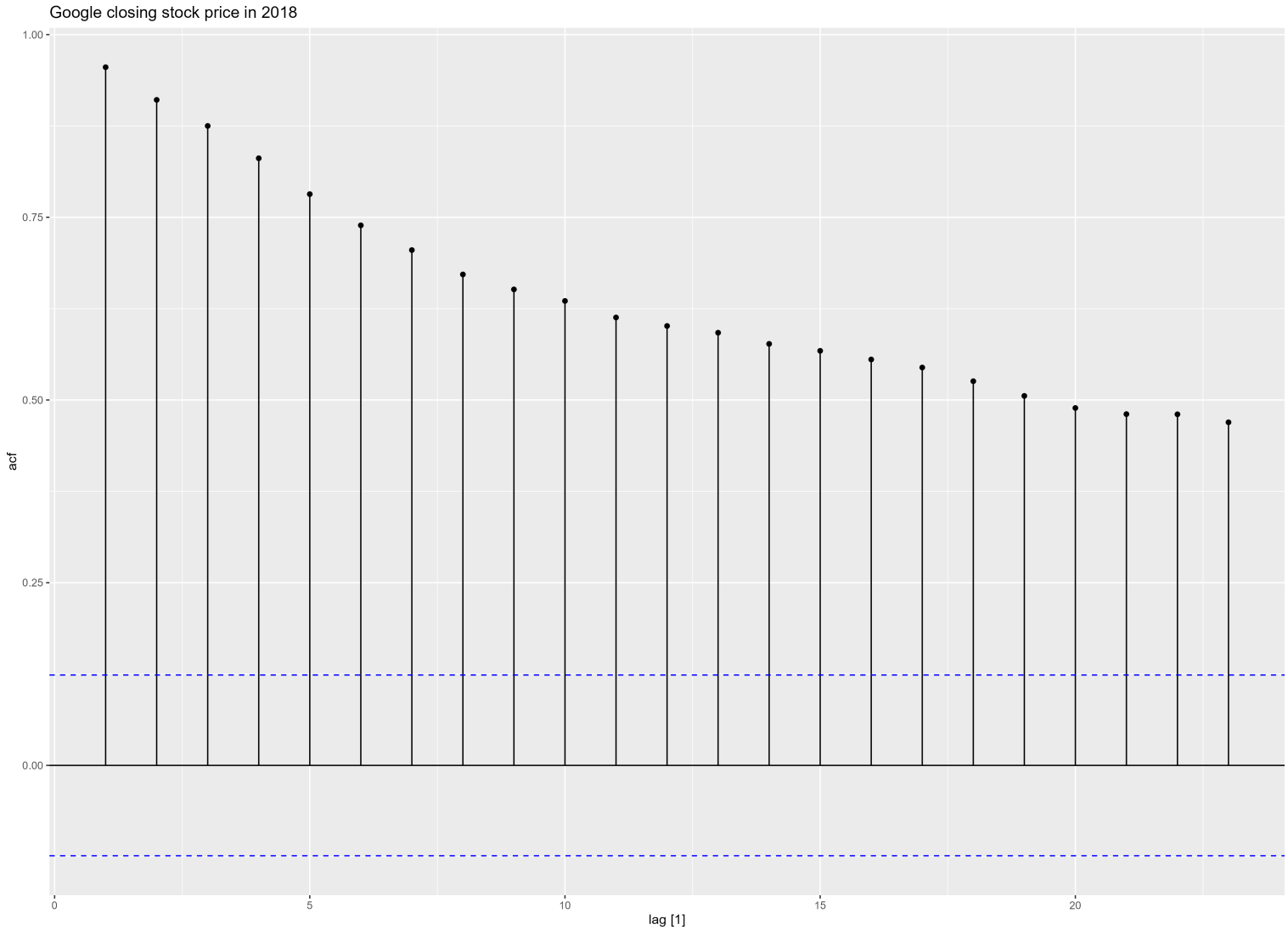
The Augmented Dickey-Fuller (ADF) test

- The ADF test is a *unit root* test.
- **The null hypothesis:** The time series is non-stationary (there is a unit root in it, which makes it non-stationary)
- **The alternative hypothesis:** The time series is stationary.
- Interpretation: If the p-value is less than the pre-defined significance level (in general 0.05) the null hypothesis is rejected and the time series is considered to be stationary.
- The ADF test can be run in R with the *adf.test* function from the *tseries* package.

Example 1

- Let us look at the closing price for Google stocks in 2018




Google closing stock price in 2018
1.00
0.75
0.50
0.25
0.00
acf
0
5
10
15
20
lag [1]

KPSS test

```
google_2018 |>
  features(Close, unitroot_kpss)
```

```
+   features(Close, unitroot_kpss)
# A tibble: 1 x 3
  Symbol kpss_stat kpss_pvalue
  <chr>       <dbl>       <dbl>
1 GOOG        0.573      0.0252
```

- The important value is the *kpss_pvalue* in the last column, which is 0.0252 (less than 0.05) $\Rightarrow$ we can reject the null hypothesis, the time series is not stationary.

ADF test

```
adf.test(google_2018$Close)
```

```
        Augmented Dickey-Fuller Test

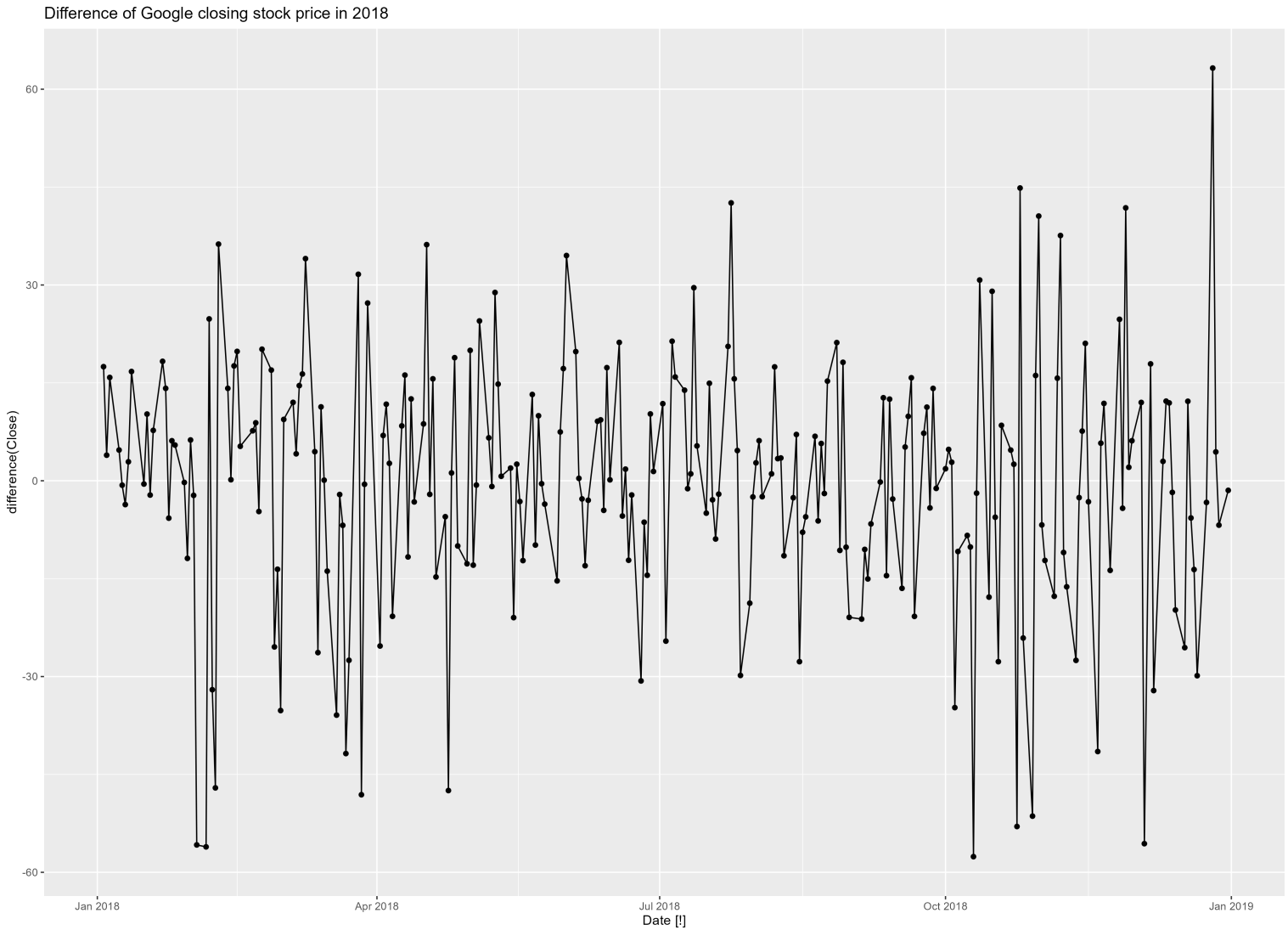
data:  google_2018$Close
Dickey-Fuller = -1.8265, Lag order = 6, p-value = 0.6489
alternative hypothesis: stationary
```

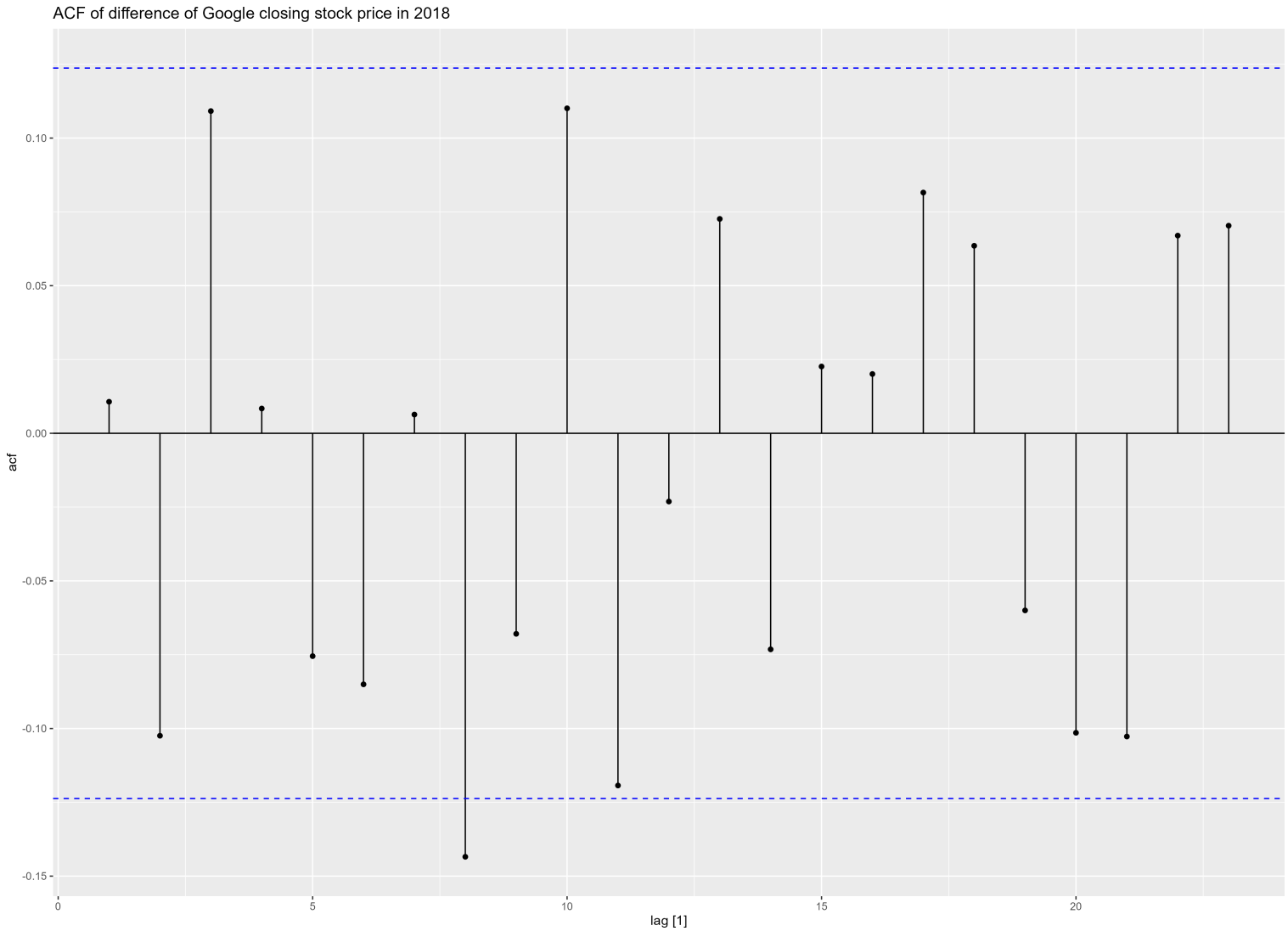
- The important value is the *p-value*, which is 0.6489 (which is greater than 0.05) $\Rightarrow$ we cannot reject the null hypothesis, the time series is not stationary.

- For this example, both the visual inspection and the statistical tests conclude that the time series is not stationary.

Example 2

- Difference of Google closing stock prices for 2018.




ACF of difference of Google closing stock price in 2018
0.10
0.05
0.00
-0.05
-0.10
-0.15
acf
0
5
10
15
20
lag [1]

KPSS test

```
# A tibble: 1 x 3
  Symbol kpss_stat kpss_pvalue
  <chr>       <dbl>       <dbl>
1 GOOG       0.0955         0.1
```

- The important value is the *kpss_pvalue* in the last column, which is 0.1 $\Rightarrow$ we cannot reject the null hypothesis (it is greater than 0.05), the time series is stationary.

ADF test

```
        Augmented Dickey-Fuller Test

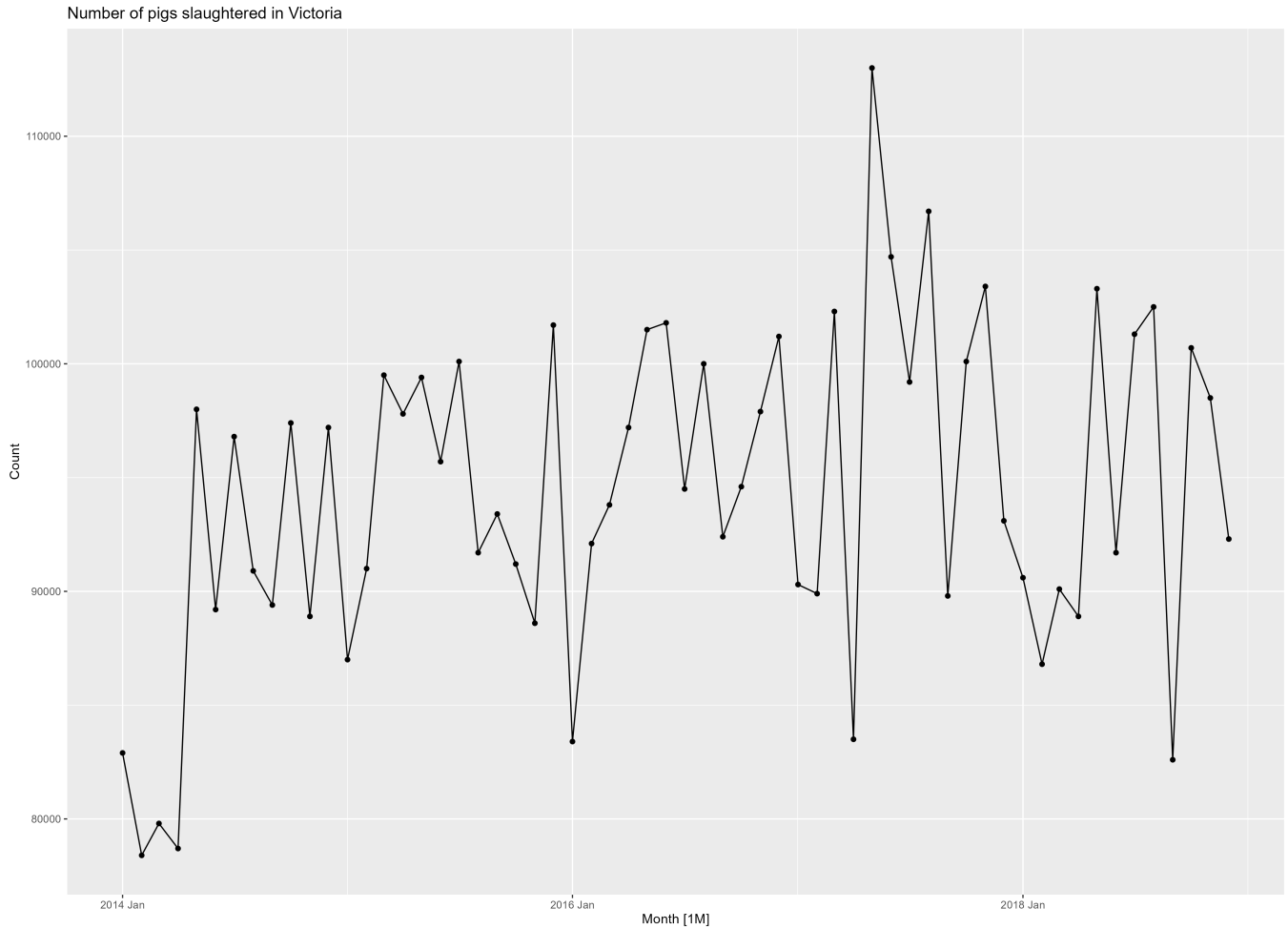
data:  na.omit(difference(google_2018$Close))
Dickey-Fuller = -6.4714, Lag order = 6, p-value = 0.01
alternative hypothesis: stationary

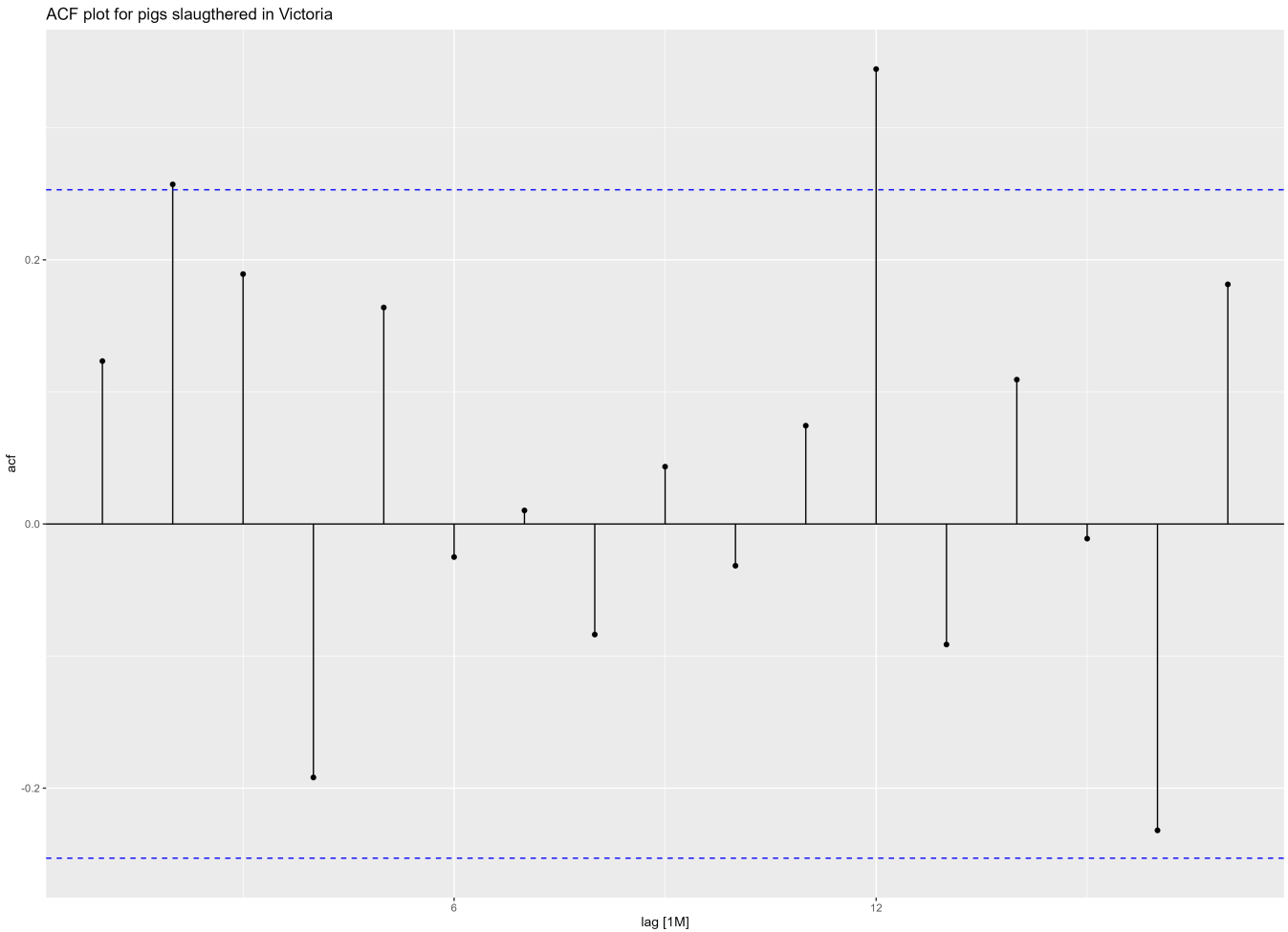
Warning message:
In adf.test(na.omit(difference(google_2018$Close))) :
  p-value smaller than printed p-value
```

- The important value is the *p-value*, which is 0.01 $\Rightarrow$ we can reject the null hypothesis, the time series is stationary.
- Actually, you can see a message that the real p-value might be smaller than the reported one, but this is not a problem.
- For this example, both the visual inspection and the statistical tests conclude that the time series is stationary.

Example 3

- Monthly total number of pigs slaughtered in Victoria, 2014-2018




ACF plot for pigs slaughtered in Victoria
acf
0.2
0.0
-0.2
6
12
lag [1M]

KPSS test

```
# A tibble: 1 × 4
  Animal State    kpss_stat kpss_pvalue
  <fct>  <fct>        <dbl>       <dbl>
1 Pigs   Victoria     0.491      0.0438
```

- The important value is the *kpss_pvalue* in the last column, which is 0.0438 $\Rightarrow$ we can reject the null hypothesis, the time series is non-stationary.

ADF test

```
        Augmented Dickey-Fuller Test

data:  aus_pigs$Count
Dickey-Fuller = -5.2471, Lag order = 3, p-value = 0.01
alternative hypothesis: stationary

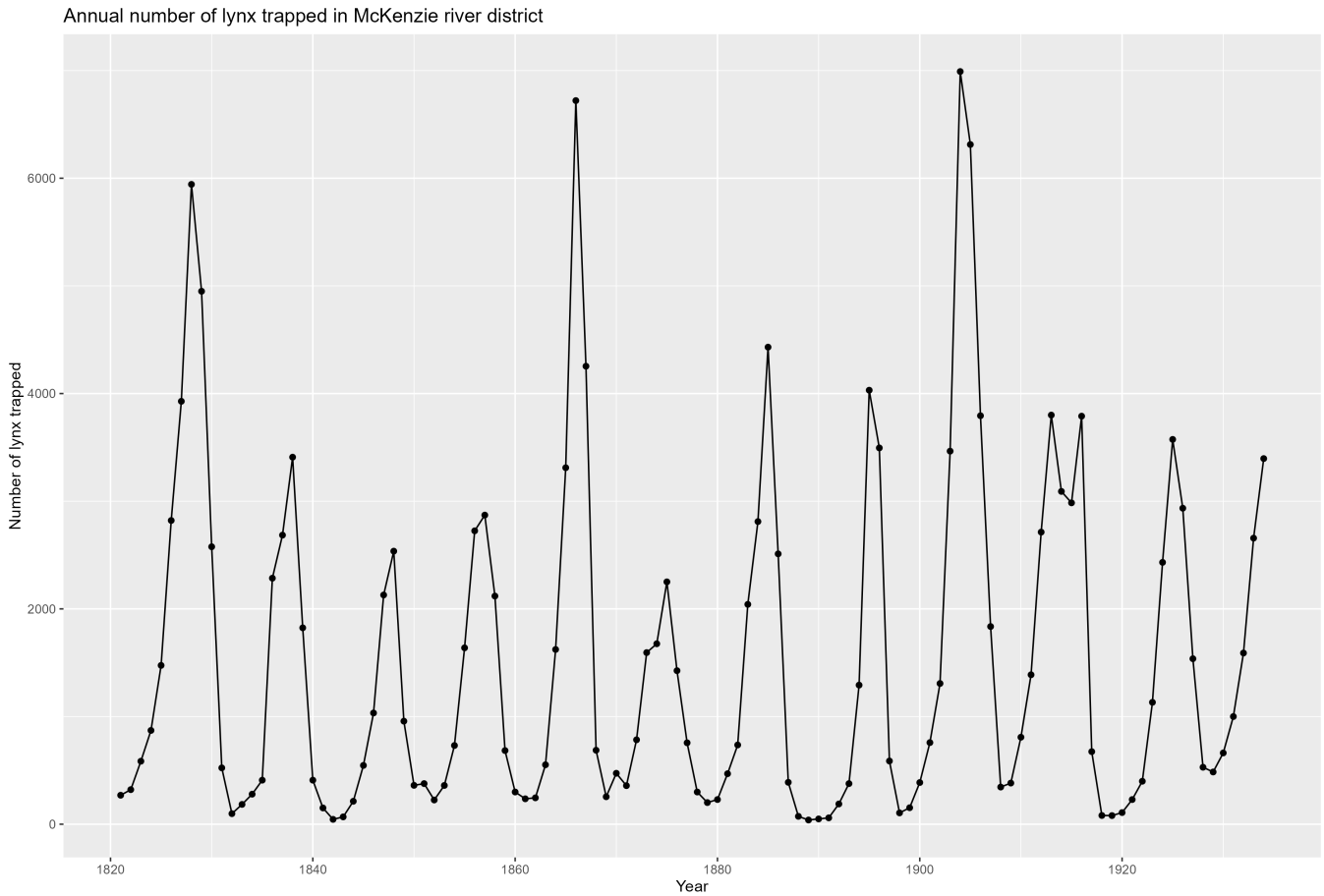
Warning message:
In adf.test(aus_pigs$Count) : p-value smaller than printed p-value
```

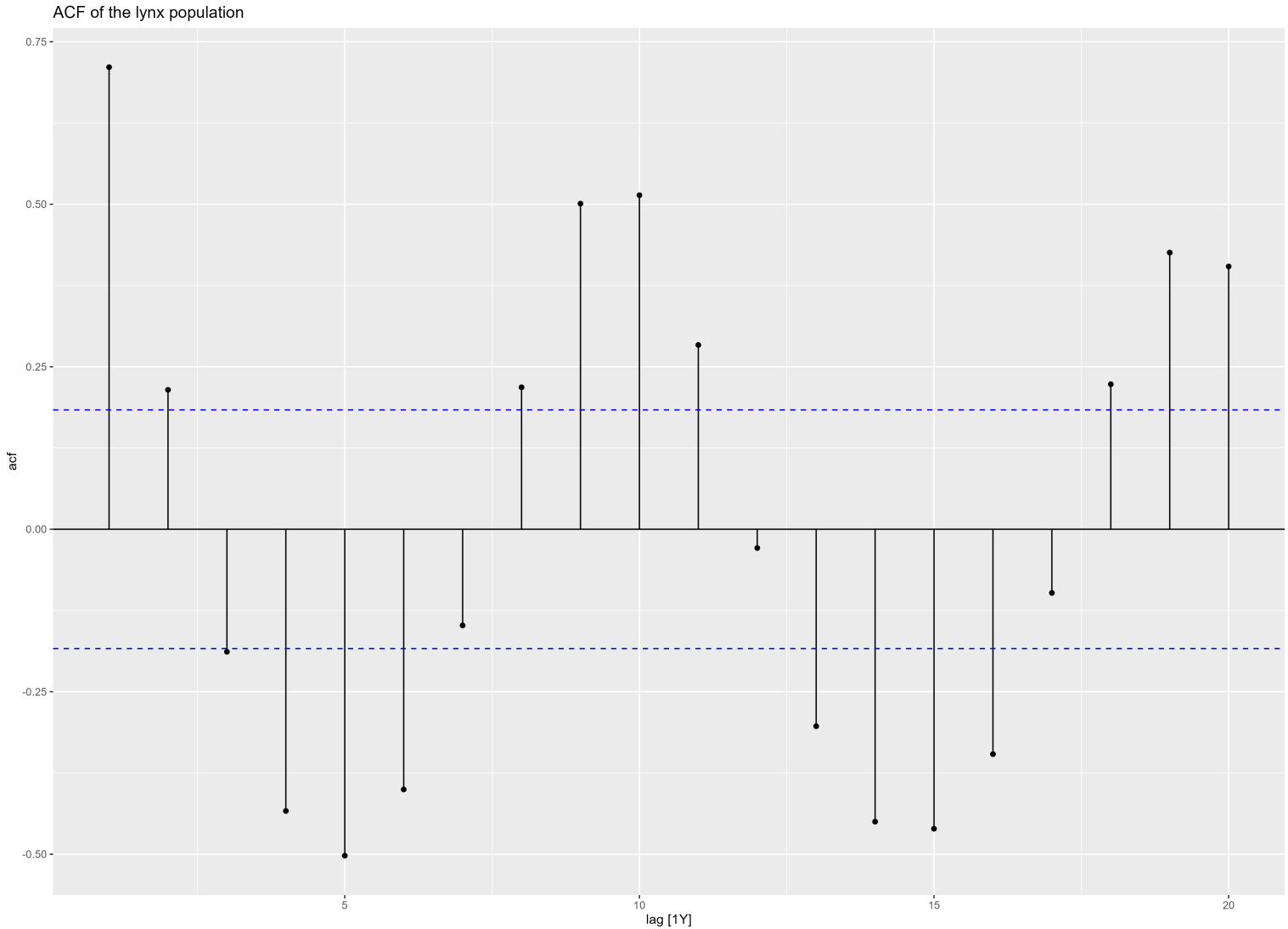
- The important value is the *p-value*, which is 0.01 $\Rightarrow$ we can reject the null hypothesis, the time series is stationary.

- When the two tests do not agree, we will consider the result of the KPSS test and we will consider that this time series is non-stationary.

Example 4

- Yearly lynx population




ACF of the lynx population
0.75
0.50
0.25
0.00
-0.25
-0.50
acf
5
10
15
20
lag [1Y]

KPSS test

```
# A tibble: 1 x 2
  kpss_stat kpss_pvalue
      <dbl>       <dbl>
1    0.0701         0.1
```

- The important value is the *kpss_pvalue* in the last column, which is 0.1 $\Rightarrow$ we cannot reject the null hypothesis (it is greater than 0.05), the time series is stationary.

ADF test

```
        Augmented Dickey-Fuller Test

data:  lynx_ts$value
Dickey-Fuller = -6.3068, Lag order = 4, p-value = 0.01
alternative hypothesis: stationary
```

- The important value is the *p-value*, which is 0.01 $\Rightarrow$ we can reject the null hypothesis, the time series is stationary.
- Both tests agree that the time series is stationary.

- If we have identified that our time series is not stationary, and we need a stationary time series, we can do the following:
  - Use a transformation to stabilize the variance (transformations will be discussed later).
  - Use differencing to stabilize the mean

Differencing

- Differencing is a transformation of the time series, we simply subtract from each observation the previous one:

$$y'_t = y_t - y_{t-1}$$

- Differencing can be repeated several times until we get a stationary time series. The number of repetitions is called the *difference order*.

- **Obs.** As in case of any transformation, if time series predictions are performed on transformed data, the result needs to be transformed back, in order to have values on the original scale.
- Assuming that you used a differenced time series and predicted y'_{t+1}, the actual prediction can be computed in the following way:

$$y'_{t+1} = y_{t+1} - y_t \quad \rightarrow \quad y_{t+1} = y'_{t+1} + y_t$$

- So, the actual prediction is the last observation plus the predicted difference.
- If multiple future points were predicted, the process is performed for each point.

- In R we can compute the difference of a time series with the *difference* function.
- Let us consider an example on the number of births in Victoria

```
?aus_births

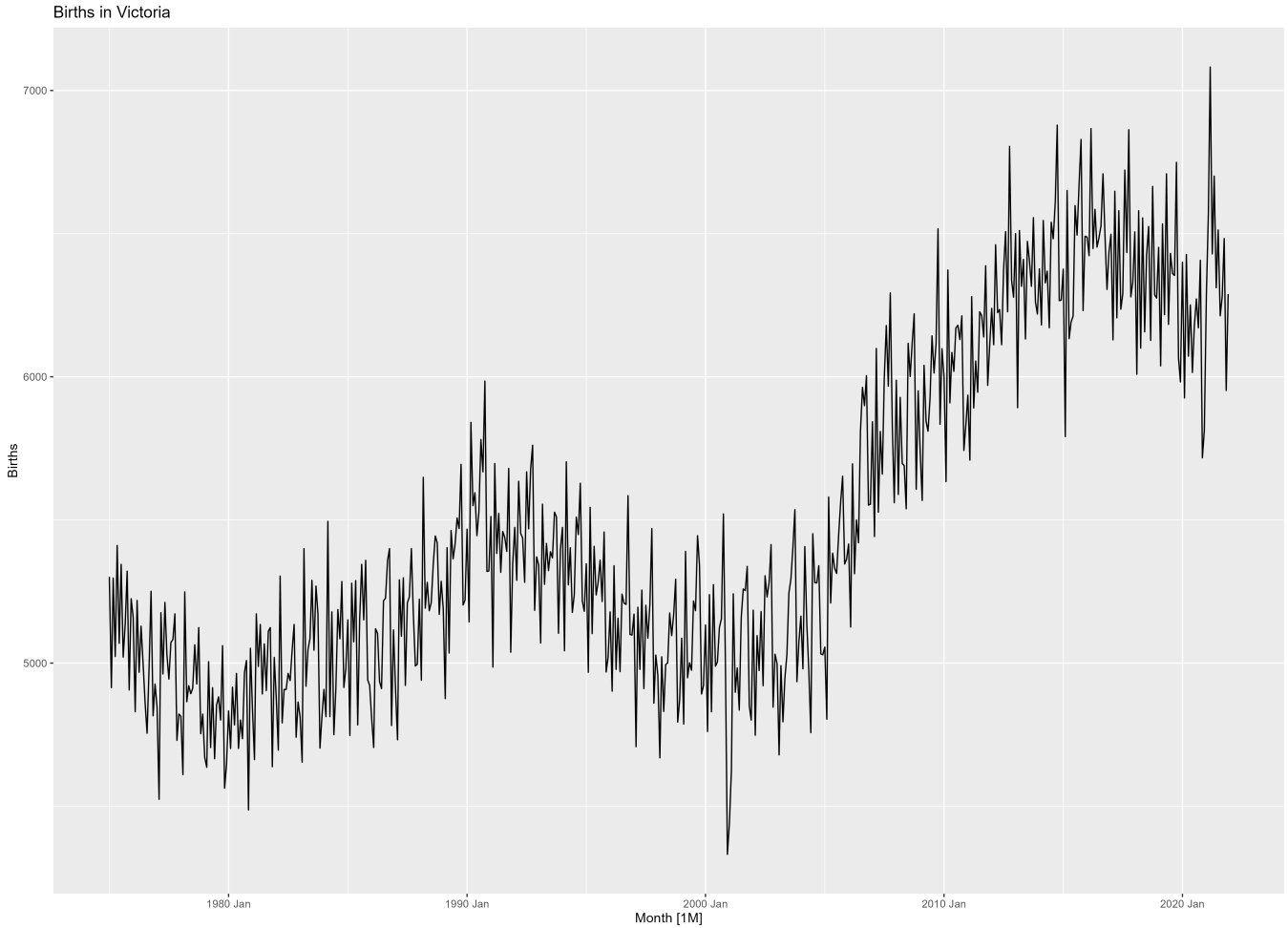
aus_births |> filter(State == "VIC") -> vic_births

vic_births |>
  autoplot() +
  labs(title = "Births in Victoria")

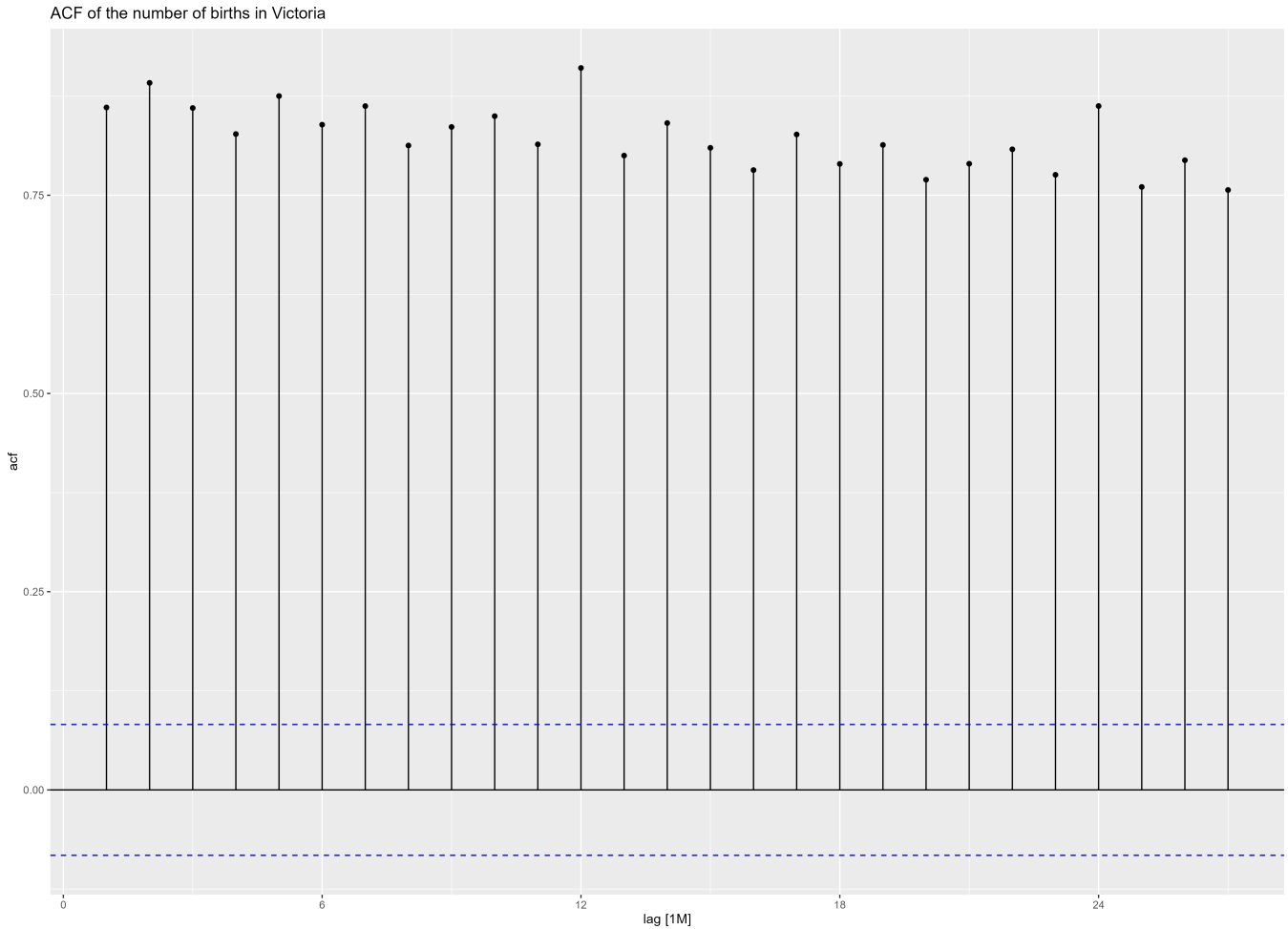
vic_births |>
  ACF() |>
  autoplot() +
  geom_point() +
  labs(title = "ACF of the number of births in Victoria")

vic_births |> features(Births, unitroot_kpss)
adf.test(vic_births$Births)
```

- Initial data (Births in Victoria state in Australia)



- The corresponding ACF plot



- And the statistical tests

```
# A tibble: 1 x 3
  State kpss_stat kpss_pvalue
  <chr>      <dbl>       <dbl>
1 VIC         6.12        0.01
```

```
        Augmented Dickey-Fuller Test

data:  vic_births$Births
Dickey-Fuller = -2.7453, Lag order = 8, p-value = 0.2628
alternative hypothesis: stationary
```

- Everything shows that the data is non-stationary.

```r
vic_births <- vic_births |>
  mutate(BirthsD = difference(Births))

vic_births

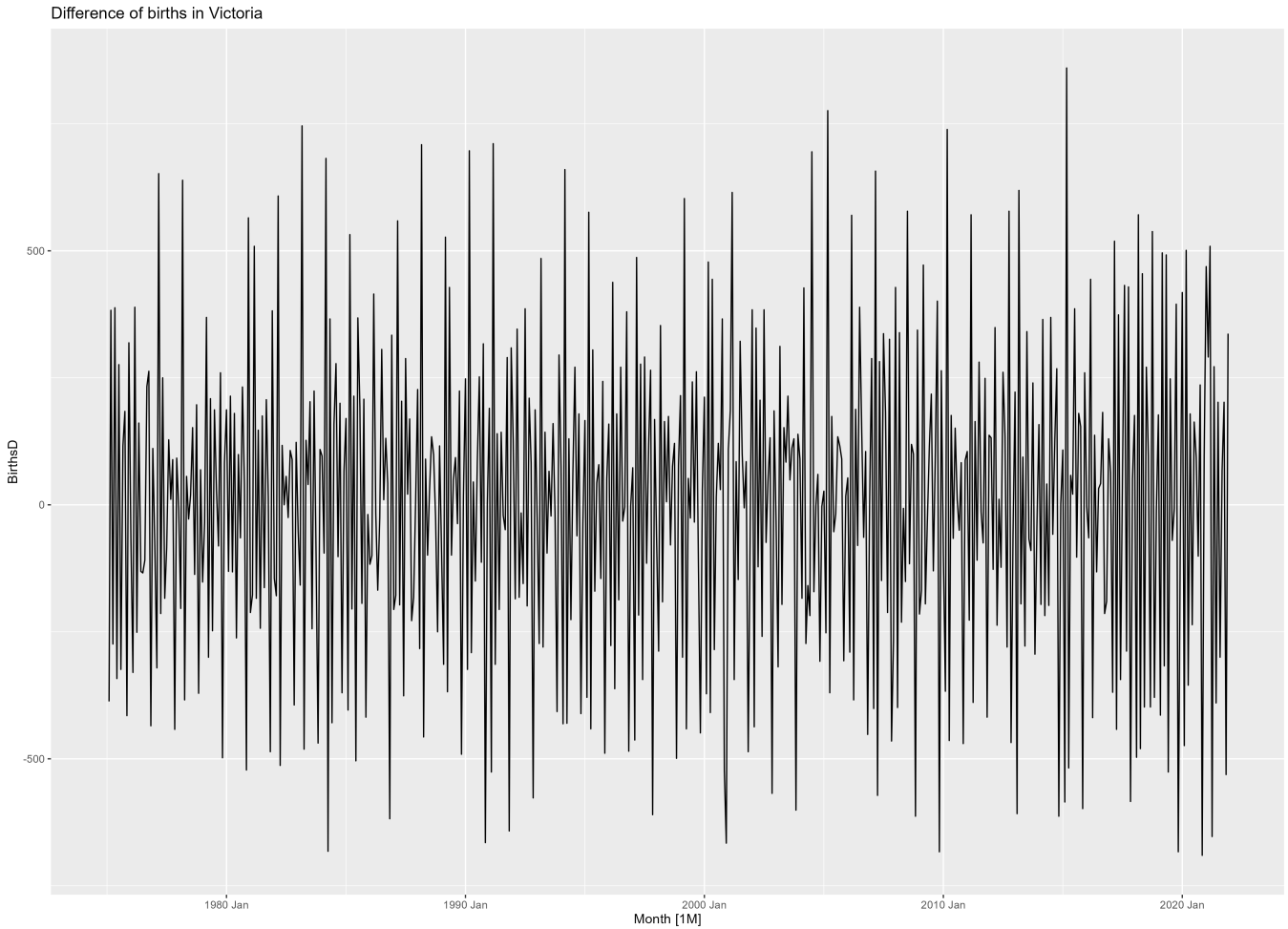
vic_births |>
  autoplot(BirthsD) +
  labs(title = "Difference of births in Victoria")

vic_births |>
  ACF(BirthsD) |>
  autoplot() +
  geom_point() +
  labs(title = "ACF of the difference of the number of births in Victoria")

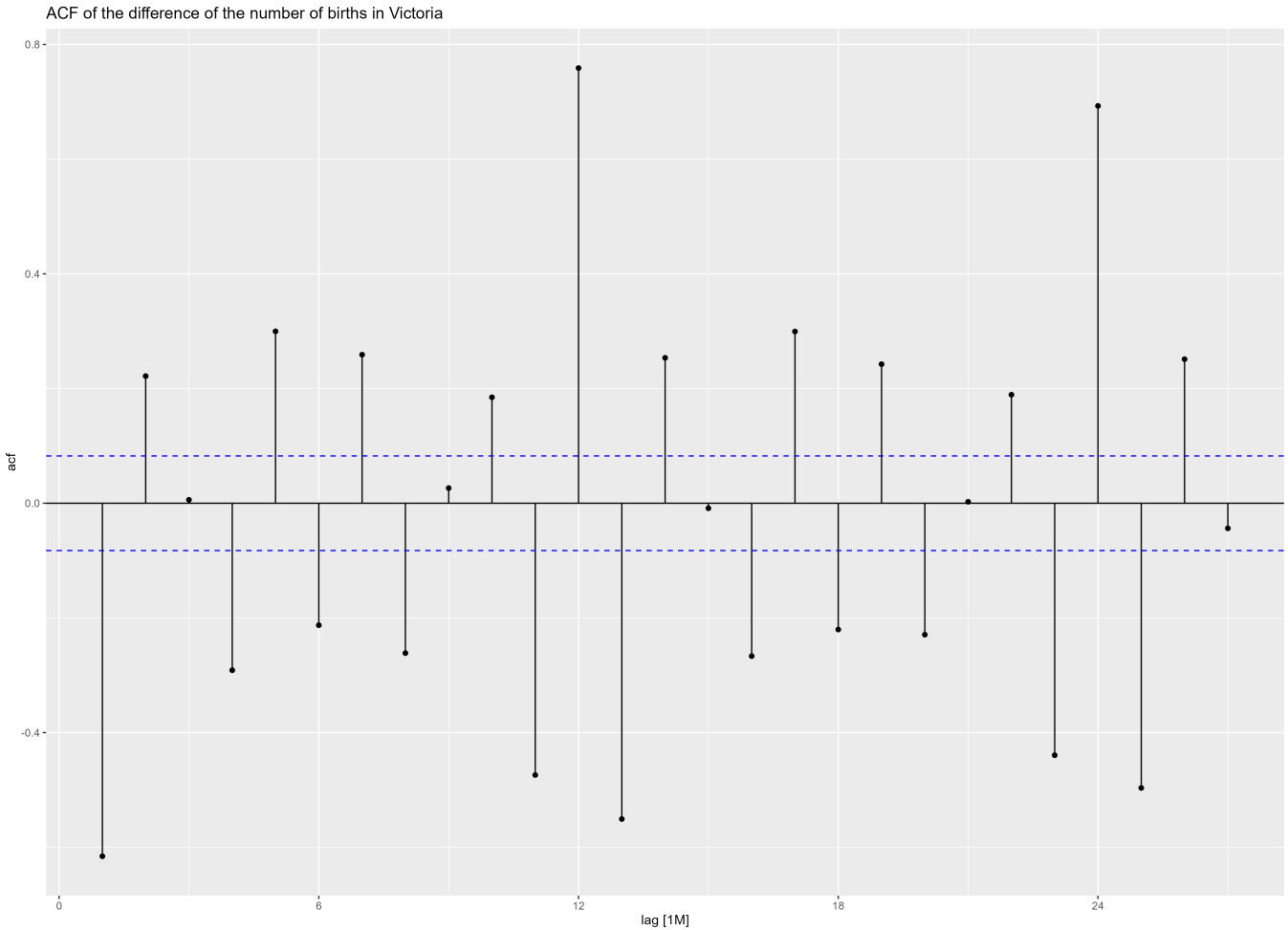
vic_births |> features(BirthsD, unitroot_kpss)

adf.test(vic_births$BirthsD |> na.omit())
```

- The differenced data



- The corresponding ACF plot



- And the statistical tests

```
# A tibble: 1 x 3
  State kpss_stat kpss_pvalue
  <chr>     <dbl>       <dbl>
1 VIC      0.0367         0.1
```

```
        Augmented Dickey-Fuller Test

data:  na.omit(vic_births$BirthsD)
Dickey-Fuller = -16.104, Lag order = 8, p-value = 0.01
alternative hypothesis: stationary

Warning message:
In adf.test(na.omit(vic_births$BirthsD)) :
  p-value smaller than printed p-value
```

- Everything shows that the data is stationary (even if it is not white noise).

Seasonal differencing

- A seasonal difference is the difference between an observation and the previous observation from the same season (so lag is equal to the length of a season, denoted by m):

$$y'_t = y_t - y_{t-m}$$

- We can do a seasonal difference with the same *difference* function in R, which takes as parameter the number of lags to consider for the difference.

```r
vic_births <- vic_births |>
  mutate(BirthsSDiff = difference(Births, 12))

vic_births |> head(n = 15)

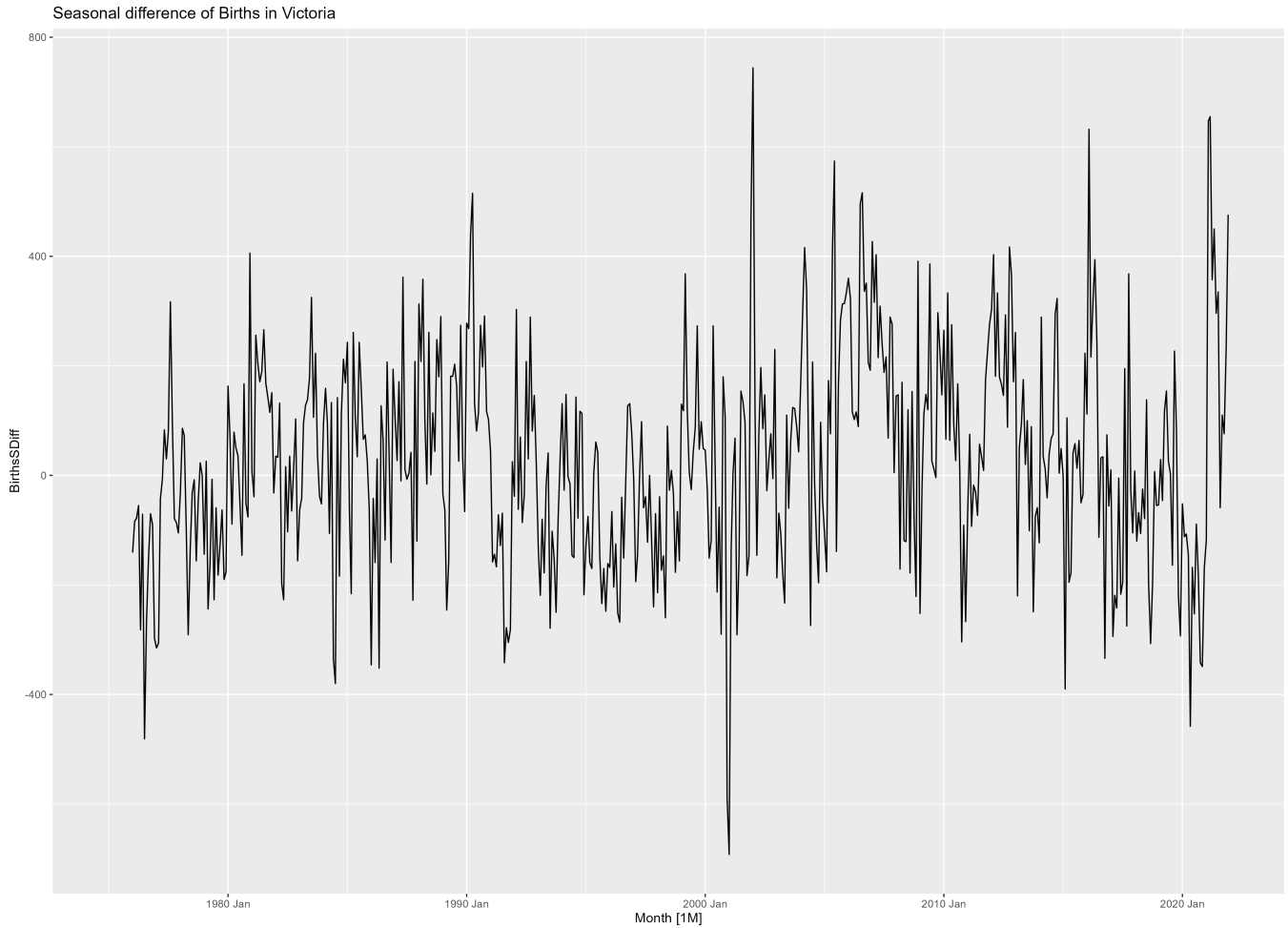
vic_births |>
  autoplot(BirthsSDiff) +
  labs(title = "Seasonal difference of Births in Victoria")

vic_births |>
  ACF(BirthsSDiff) |>
  autoplot() +
  geom_point() +
  labs(title = "ACF of the seasonal difference of the number of births in
      Victoria")

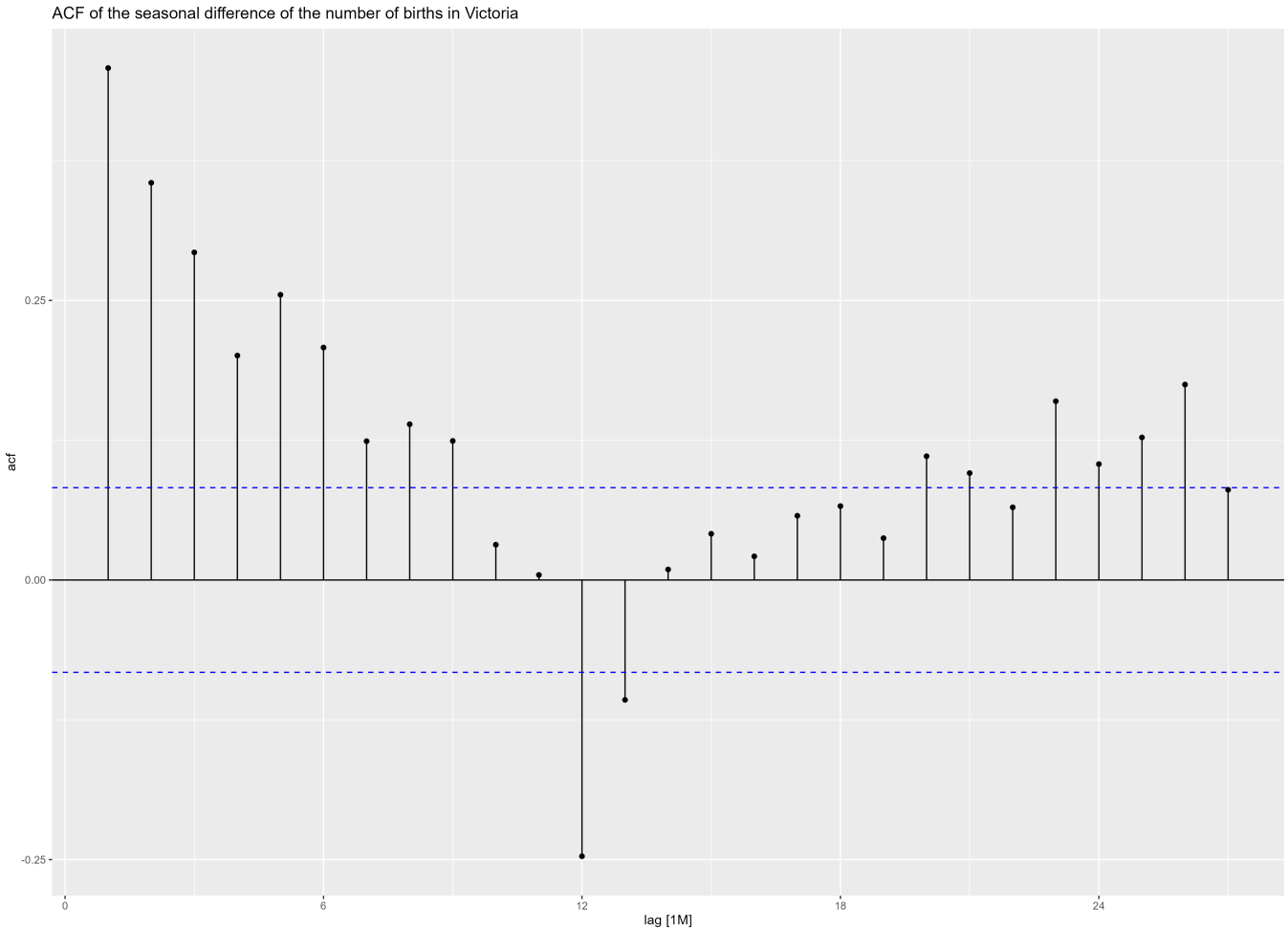
vic_births |> features(BirthsSDiff, unitroot_kpss)

adf.test(vic_births$BirthsSDiff |> na.omit())
```

- The seasonally differenced data



- The corresponding ACF plot



- And the statistical tests

```
# A tibble: 1 × 3
  State kpss_stat kpss_pvalue
  <chr>      <dbl>       <dbl>
1 VIC        0.427      0.0656
```

```
        Augmented Dickey-Fuller Test

data:  na.omit(vic_births$BirthsSDiff)
Dickey-Fuller = -5.3991, Lag order = 8, p-value = 0.01
alternative hypothesis: stationary

Warning message:
In adf.test(na.omit(vic_births$BirthsSDiff)) :
  p-value smaller than printed p-value
```

- Everything shows that the data is stationary (even if it is not white noise).

Beer production

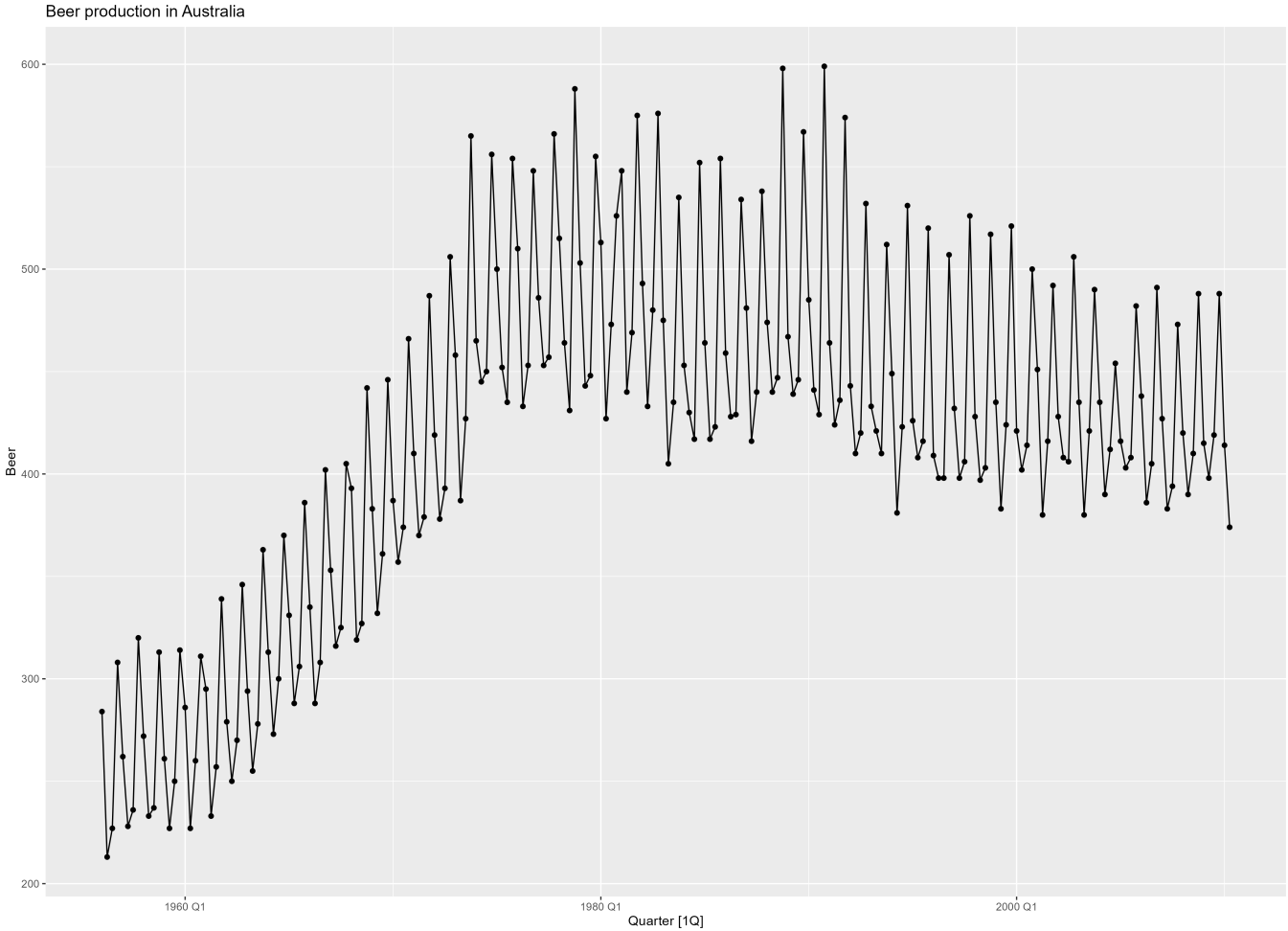
- Let us consider another example, the quarterly beer production in Australia (from now on we will only look at the KPSS test).

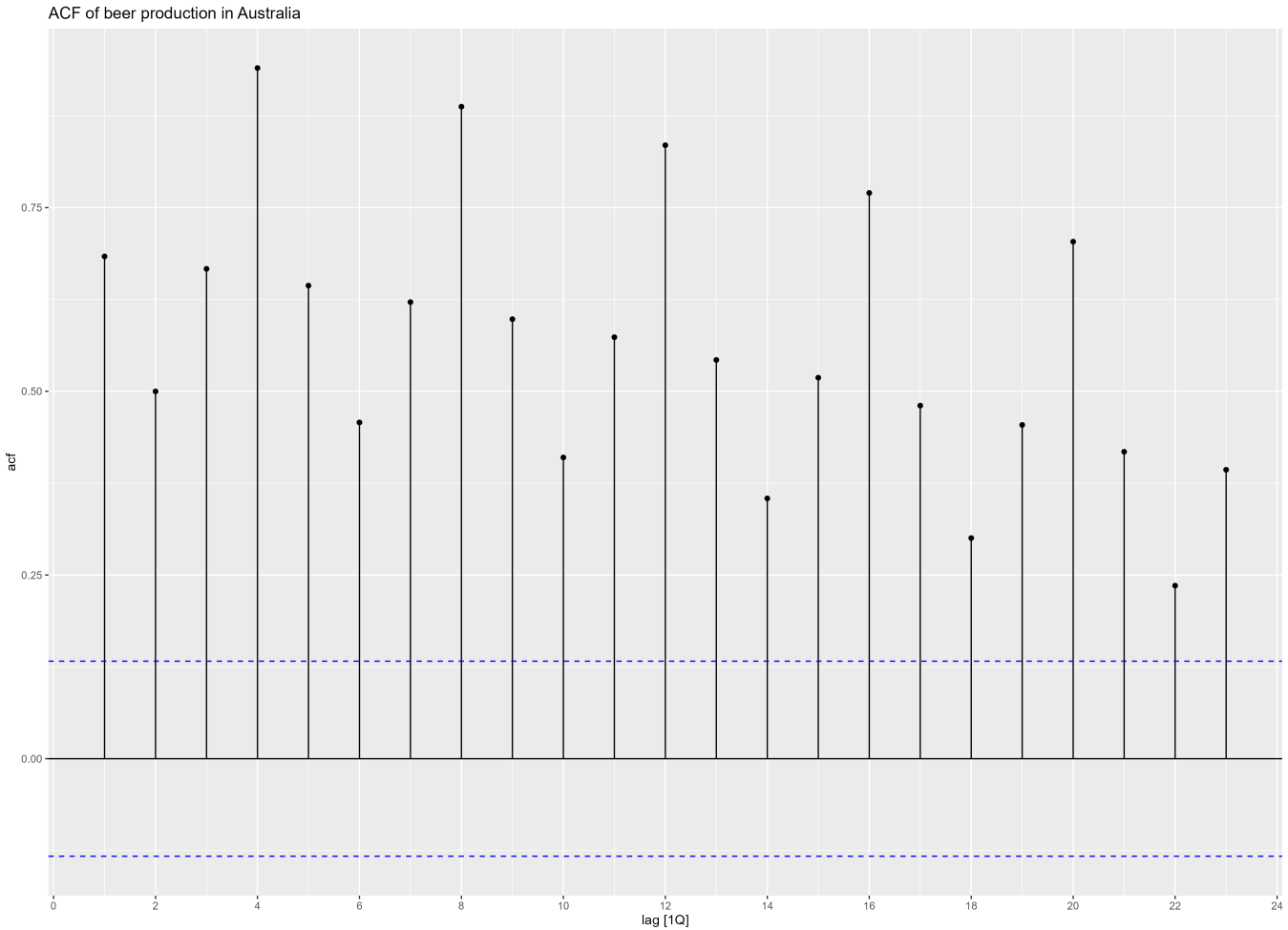
- Sometimes both a seasonal and a regular (first order) difference is needed to make the data stationary. Their order is not important, but it is suggested to start with the seasonal differencing, maybe the other one is not going to be needed after that.

```
beer |> autoplot() +
  geom_point() +
  labs(title = "Beer production in Australia")

beer |>
  ACF() |>
  autoplot() +
  geom_point() +
  labs(title = "ACF of beer production in Australia")

beer |>
  features(Beer, unitroot_kpss)
```


Beer production in Australia
600
500
400
300
200
Beer
1960 Q1
1980 Q1
2000 Q1
Quarter [1Q]


ACF of beer production in Australia
acf
0.75
0.50
0.25
0.00
0
2
4
6
8
10
12
14
16
18
20
22
24
lag [1Q]

```
# A tibble: 1 x 2
  kpss_stat kpss_pvalue
      <dbl>       <dbl>
1      2.20        0.01
```

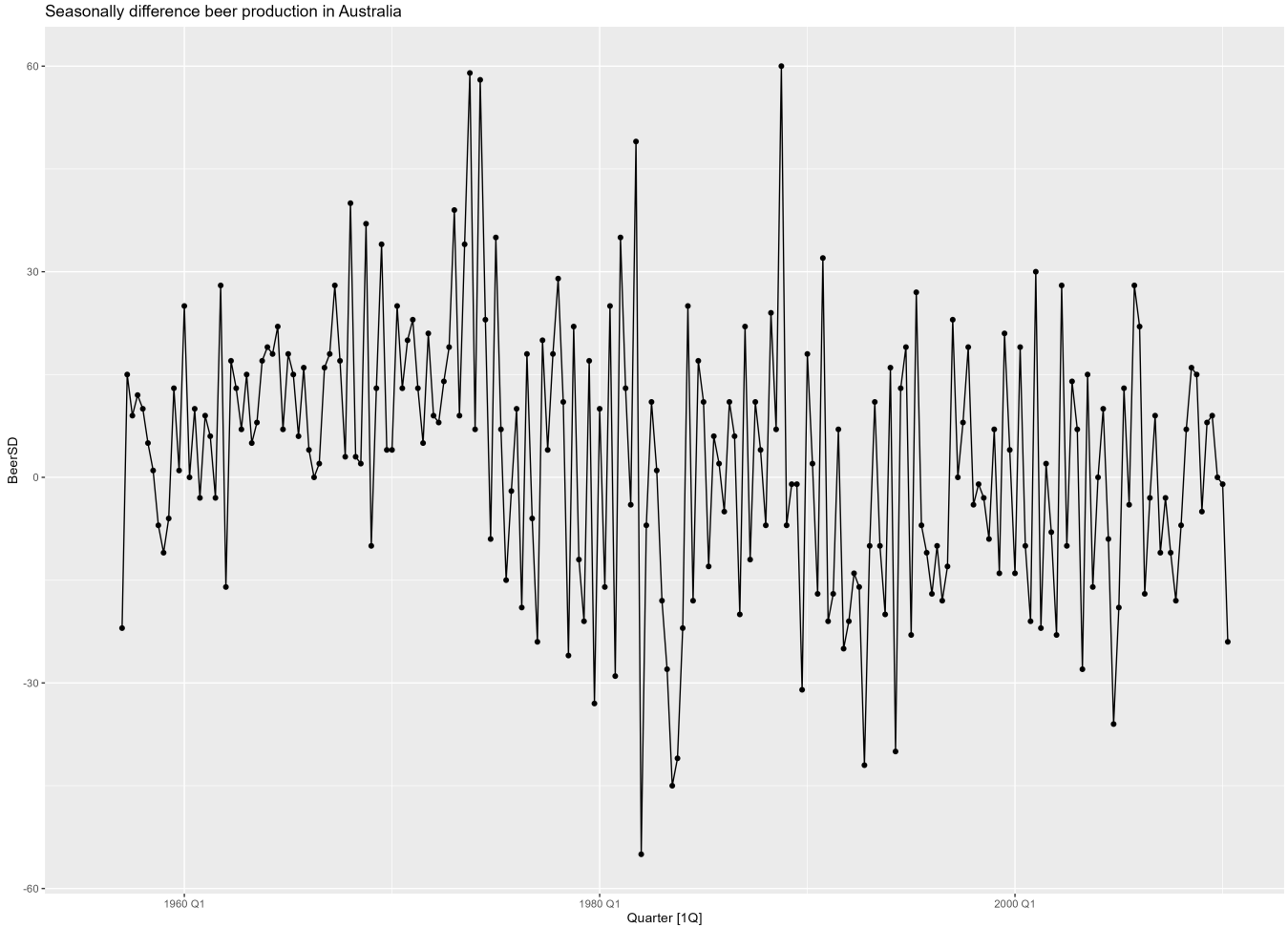
- Clearly not stationary.

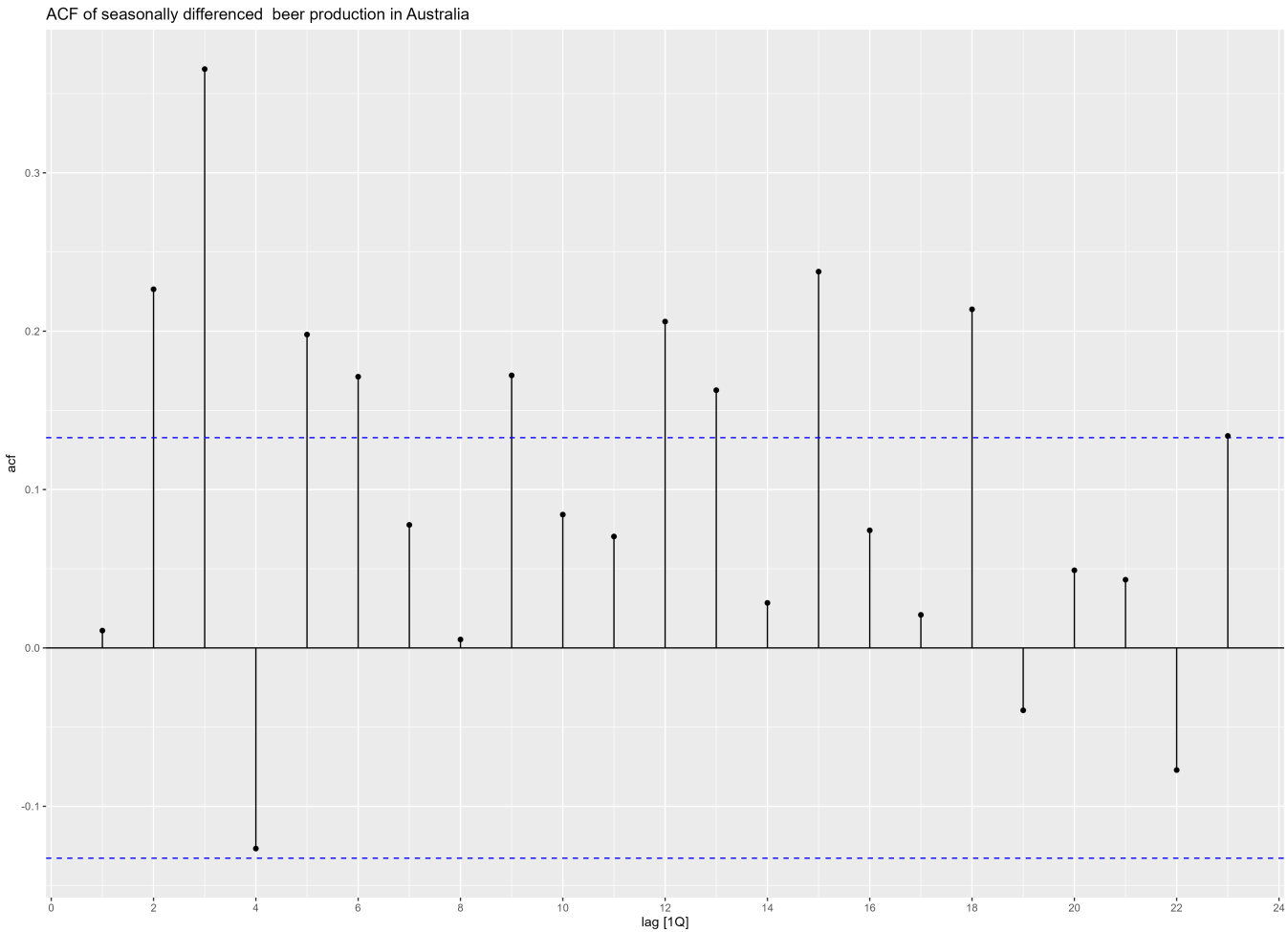
```r
beer <- beer |> mutate(BeerSD = difference(Beer, 4))

beer |> autoplot(BeerSD) +
  geom_point() +
  labs(title = "Seasonally difference beer production in Australia")

beer |>
  ACF(BeerSD) |>
  autoplot() +
  geom_point() +
  labs(title = "ACF of seasonally differenced  beer production in Australia")

beer |>
  features(BeerSD, unitroot_kpss))
```


Seasonally difference beer production in Australia
BeerSD
60
30
0
-30
-60
1960 Q1
1980 Q1
2000 Q1
Quarter [1Q]


ACF of seasonally differenced beer production in Australia
acf
0.3
0.2
0.1
0.0
-0.1
0
2
4
6
8
10
12
14
16
18
20
22
24
lag [1Q]

```
# A tibble: 1 x 2
  kpss_stat kpss_pvalue
      <dbl>       <dbl>
1      1.21        0.01
```

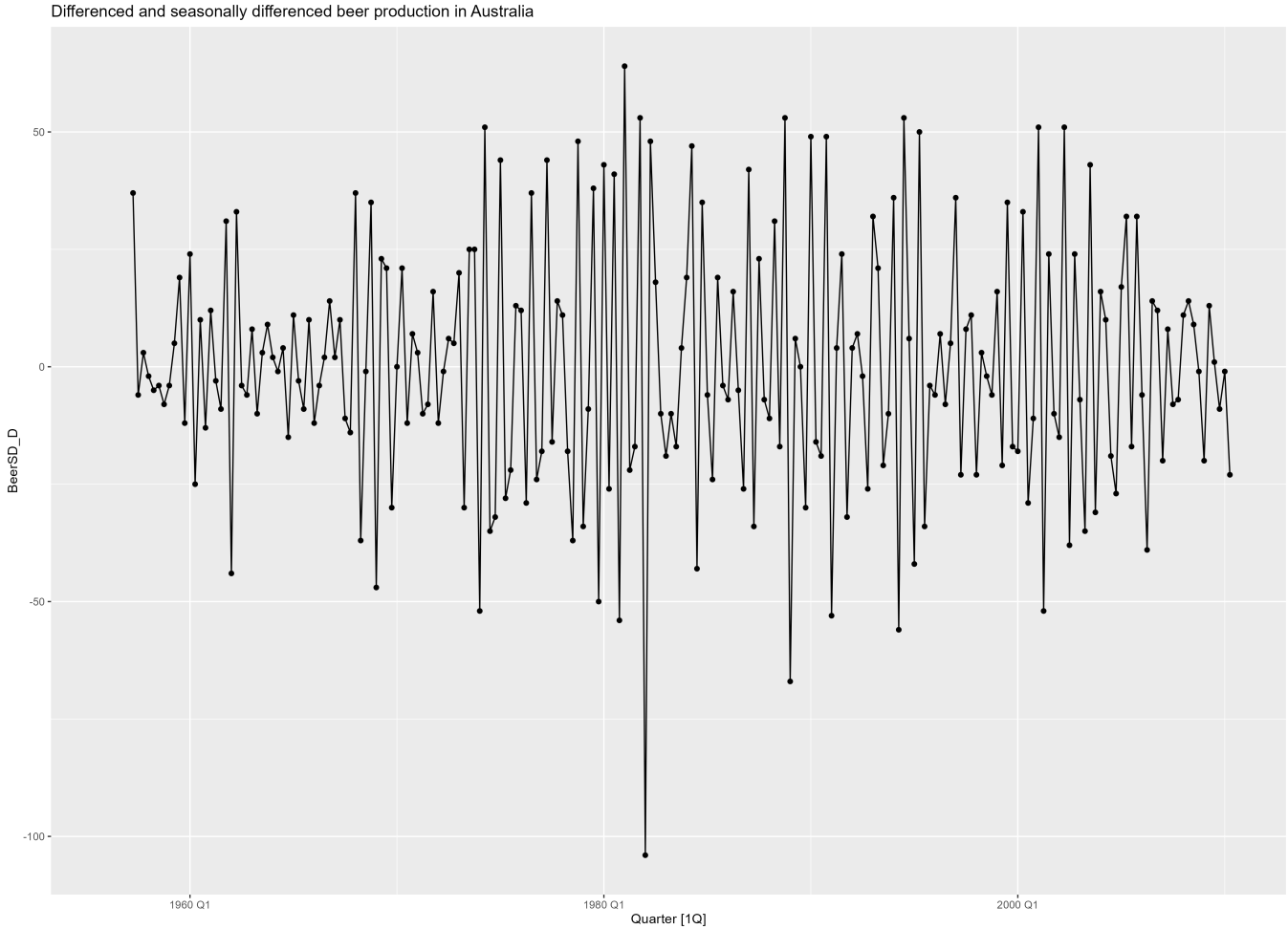
- Still not stationary

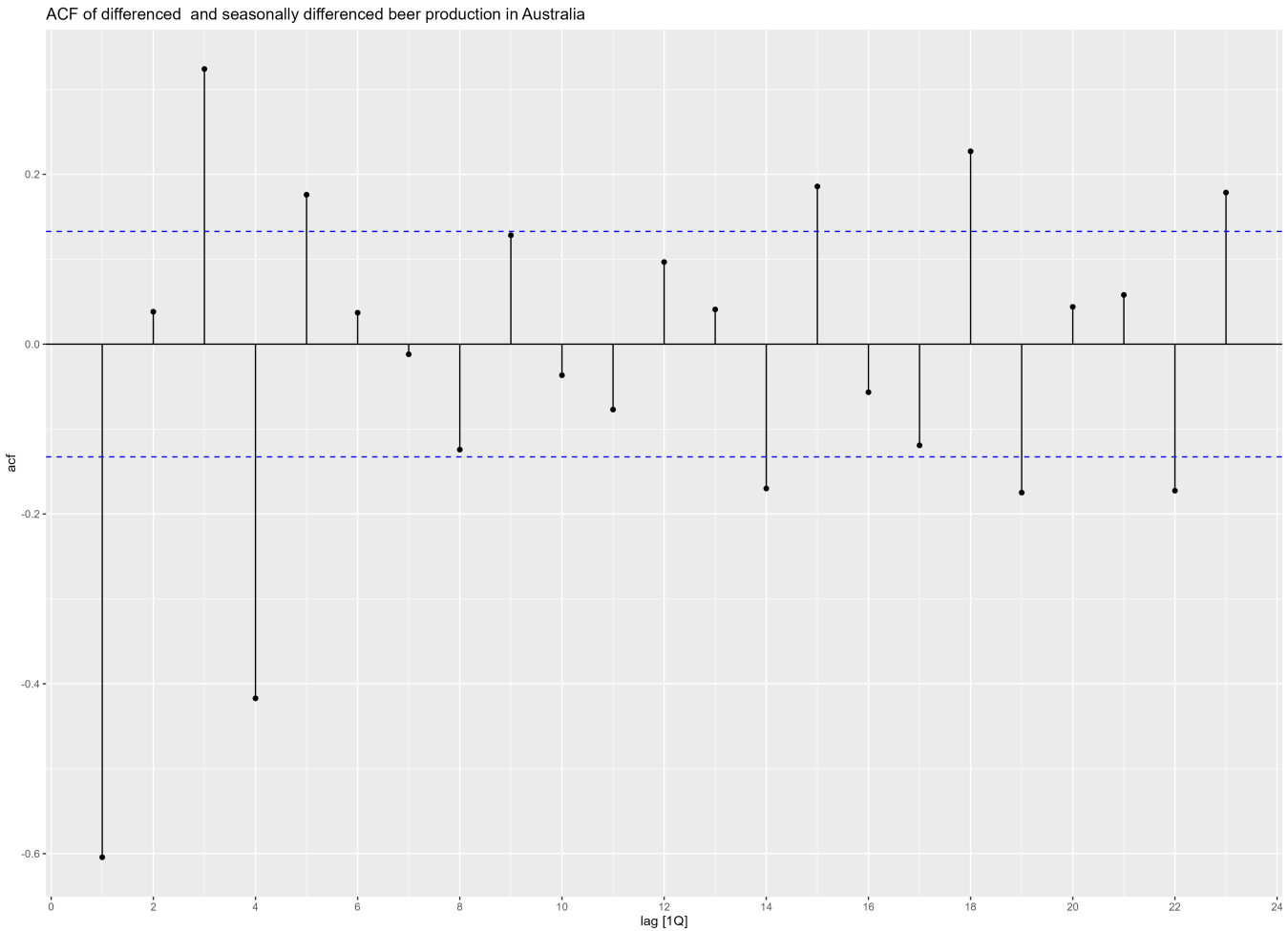
```r
beer <- beer |>
  mutate(BeerSD_D = difference(BeerSD, 1))

beer |> autoplot(BeerSD_D) +
  geom_point() +
  labs(title = "Differenced and seasonally differenced beer production in
      Australia")

beer |>
  ACF(BeerSD_D) |>
  autoplot() +
  geom_point() +
  labs(title = "ACF of differenced  and seasonally differenced beer production
      in Australia")

beer |>
  features(BeerSD_D, unitroot_kpss)
```


Differenced and seasonally differenced beer production in Australia
BeerSD_D
50
0
-50
-100
1960 Q1
1980 Q1
2000 Q1
Quarter [1Q]


ACF of differenced and seasonally differenced beer production in Australia
acf
0.2
0.0
-0.2
-0.4
-0.6
0
2
4
6
8
10
12
14
16
18
20
22
24
lag [1Q]

```
# A tibble: 1 x 2
  kpss_stat kpss_pvalue
      <dbl>       <dbl>
1    0.0399         0.1
```

- Now it is stationary.

- For the previous examples, we followed a general flow: check the data for stationarity (for example with the KPSS test), difference if it is not stationary, check again, difference again, etc.
- This process can be simplified by the use of *unitroot_ndiffs* function, which will tell us how many times a time series should be differenced to become stationary.
- Similarly, the *unitroot_nsdiffs* function will tell us how many times do we need to perform a seasonal differencing on the data.

- Let's see how these functions work using the Beer time series.

```
beer |> features(Beer, unitroot_ndiffs)
beer |> features(Beer, unitroot_nsdiffs)
beer |> features(Beer, c(unitroot_ndiffs, unitroot_nsdiffs))
```

```
> beer |> features(Beer, unitroot_ndiffs)
# A tibble: 1 x 1
  ndiffs
   <int>
1      1
> beer |> features(Beer, unitroot_nsdiffs)
# A tibble: 1 x 1
  nsdiffs
    <int>
1       1
> beer |> features(Beer, c(unitroot_ndiffs, unitroot_nsdiffs))
# A tibble: 1 x 2
  ndiffs nsdiffs
   <int>   <int>
1      1       1
```

- The results show that we need one seasonal and one first order differencing.

- The two functions (*unitroot_ndiffs* and *unitroot_nsdiffs*) measure the required number of differencing independently of each other. Sometimes, it might happen that after performing one differencing (start with the seasonal), the second is no longer needed.

```
vic_births |> features(Births, c(unitroot_ndiffs, unitroot_nsdiffs))

vic_births |> features(difference(Births, 12), c(unitroot_ndiffs, unitroot_
    nsdiffs))

vic_births |> features(difference(Births, 1), c(unitroot_ndiffs, unitroot_
    nsdiffs))
```

```
> vic_births |> features(Births, c(unitroot_ndiffs, unitroot_nsdiffs))
# A tibble: 1 × 3
  State ndiffs nsdiffs
  <chr>  <int>   <int>
1 VIC        1       1
> vic_births |> features(difference(Births, 12), c(unitroot_ndiffs, unitroot_nsdiffs))
# A tibble: 1 × 3
  State ndiffs nsdiffs
  <chr>  <int>   <int>
1 VIC        0       0
> vic_births |> features(difference(Births, 1), c(unitroot_ndiffs, unitroot_nsdiffs))
# A tibble: 1 × 3
  State ndiffs nsdiffs
  <chr>  <int>   <int>
1 VIC        0       1
```

- Let us look again at the Google stock data.
- The initial data is clearly not stationary, it shows a *wandering behaviour* which is specific for financial and economic data.

```
google_2018 <- gafa_stock |>
  filter(Symbol == "GOOG", year(Date) == 2018) |>
  select(Date, Close)

google_2018
google_2018 |> autoplot() + geom_point()
google_2018 |> ACF() |>
  autoplot() + geom_point()
```

- If we difference it, we get a stationary time series.
- The differenced data has values in the [-57.6, 63.2] interval and its mean value is 0.12.
- Moreover, besides being stationary, the differenced data is also a white noise.

```
google_2018 |> autoplot(difference(Close)) + geom_point()
google_2018 |> features(difference(Close), unitroot_kpss)

google_2018 |> ACF(difference(Close)) |>
  autoplot() + geom_point()

#first value after difference is NA. max, min, mean cannot handle it
diff <- difference(google_2018$Close) |> tail(250)
diff
max(diff)
min(diff)
mean(diff)
```

Random walk model

- A differenced time series is the *change* between consecutive observations: $y'_t = y_t - y_{t-1}$. The resulting series will have one less observation, because we do not have y'_1.
- Since the differenced time series is white noise, we can conclude that the daily change in stock price is a random amount, uncorrelated with the previous day.
- When the differenced time series is white noise with 0 mean, we can write this as: $y_t - y_{t-1} = \epsilon_t$ (where ϵ_t denotes the white noise, assumed to be normally and independently distributed with mean 0)
- If we rearrange the terms, we have the *random walk* model: $y_t = y_{t-1} + \epsilon_t$
- This means that every observation is equal to the previous one plus a random noise.

- In case of the random walk model, the best forecasting method is to simply forecast observation $y_{T+1|T} = y_T$, since the expected value of the error term is zero.